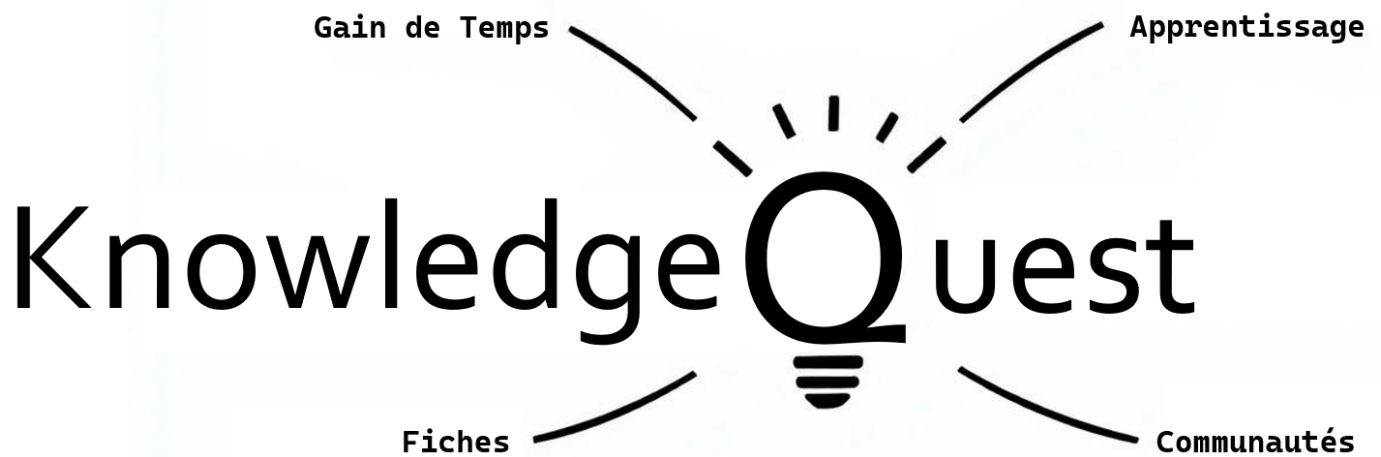




Projet JavaScript – A2MSI S2

Mouad Bentefrit – Sofiane Bennedjima – Bastien Massa



Table des matières

1. INTRODUCTION ET PRESENTATION DU PROJET	2
CONTEXTE ET PROBLEMATIQUE	2
OBJECTIFS DU PROJET.....	3
PRESENTATION DE L'IDEE DU PROJET	6
2. ANALYSE ET CONCEPTION	8
ÉTUDE DES BESOINS	8
MODELE CONCEPTUEL.....	9
JUSTIFICATION DES CHOIX TECHNIQUES	11
3. DEVELOPPEMENT ET IMPLEMENTATION	13
DESCRIPTION DU DEVELOPPEMENT	13
METHODOLOGIE UTILISEE	ERREUR ! SIGNET NON DEFINI.
METHODES ET OUTILS UTILISES.....	14
DECOMPOSITION DU PROJET ET REPARTITION DES TACHES	16
4. TESTS ET VALIDATION	39
STRATEGIE DE TEST	39
RESULTATS OBTENUS.....	39
IDENTIFICATION ET CORRECTION DES ERREURS	40
AMELIORATIONS APPORTEES SUITE AUX TESTS.....	40
5. BILAN ET PERSPECTIVES.....	40
BILAN DU TRAVAIL.....	40
DEFIS RELEVES	41
INTERET DU PROJET POUR L'APPRENTISSAGE.....	41
DIFFICULTES RENCONTREES.....	41
LEÇONS APPRISSES	41
PERSPECTIVES D'AMELIORATION	42
ÉVOLUTIONS POSSIBLES DU PROJET	42
6. CONCLUSION	42
CONCLUSION	42
ANNEXES	ERREUR ! SIGNET NON DEFINI.



1. Introduction et Présentation du Projet

Contexte et Problématique

Dans le cadre de notre formation d'ingénieur en alternance, nous avons pour mission de participer à des projets concrets, en lien avec les attentes du monde professionnel. Ce projet s'inscrit dans une démarche pédagogique visant à renforcer nos compétences techniques, organisationnelles et collaboratives à travers une mise en situation réaliste. Il nous permet de mettre en pratique les savoirs acquis au cours de notre formation, notamment dans les domaines du développement web, de la base de données notamment NoSQL et de la gestion de projet.

Le contexte académique dans lequel s'inscrit ce projet est particulièrement stimulant car il combine l'autonomie propre aux travaux pratiques avec des exigences réelles en matière de qualité de code, de documentation, de respect des délais et d'atteinte des objectifs. En choisissant de développer une application web full-stack en JavaScript, nous avons relevé le défi de concevoir un système complet, de l'interface utilisateur jusqu'à la base de données, tout en assurant une communication fluide via des API REST.

Ce cadre académique, allié à notre expérience en entreprise dans le cadre de l'alternance, nous a permis de concevoir un projet à la fois ambitieux, pertinent et formateur.

Le besoin identifié repose sur un constat clair et fréquent dans le milieu universitaire : les étudiants, quelle que soit leur filière, doivent faire face à une quantité importante de contenus à assimiler dans des délais souvent très courts. Cette surcharge de travail les pousse à chercher des méthodes de révision plus efficaces, mais la création manuelle de supports d'entraînement, comme les questionnaires à choix multiples (QCM), demande elle-même du temps et des efforts considérables.

De nombreux étudiants renoncent ainsi à cette méthode pourtant bénéfique, faute de moyens pour la mettre en œuvre rapidement. À partir de ce constat, nous avons choisi de développer une application capable d'automatiser ce processus tout en conservant une dimension personnalisable et intuitive. L'objectif est de permettre aux utilisateurs de générer des QCM directement à partir de leurs propres documents de cours, avec le soutien d'une intelligence artificielle capable de comprendre le contenu, d'en extraire l'essentiel et de produire des questions pertinentes. Ce système vise non seulement à faire gagner du temps, mais aussi à renforcer la qualité de l'apprentissage en proposant des exercices ciblés et adaptés au contenu réel des cours suivis.



Objectifs du Projet

L'objectif principal du projet est de créer une application web interactive permettant aux étudiants de transformer automatiquement leurs documents de cours (au format PDF ou Word) en questionnaires à choix multiples, grâce à l'intégration d'une API d'intelligence artificielle.

Ce projet vise plusieurs objectifs pédagogiques et techniques :

- Objectifs Techniques :

Maîtriser le développement d'une architecture web complète signifie être capable de concevoir, implémenter et faire interagir de manière fluide les différentes couches d'une application moderne.

- Cela commence par le **frontend**, c'est-à-dire l'interface utilisateur, qui doit être à la fois ergonomique, esthétique et responsive. Cette partie est développée en HTML, CSS et JavaScript, et dans notre projet, elle gère notamment l'envoi des fichiers, l'affichage des QCM générés, la navigation utilisateur et le retour des résultats.
- Le **backend**, quant à lui, assure la gestion de la logique métier. Il reçoit les requêtes du frontend, traite les données, effectue les appels nécessaires à des services tiers (comme notre API d'intelligence artificielle), et renvoie les réponses attendues. Dans notre cas, ce backend est basé sur Node.js et utilise le framework Express.js, qui facilite la gestion des routes API REST et des échanges de données asynchrones.
- La **base de données** est le cœur de la persistance des données. Nous avons opté pour MongoDB, un système NoSQL parfaitement adapté aux structures souples et évolutives comme les QCM, les utilisateurs, ou les historiques de résultats. Cette base nous permet de stocker et retrouver efficacement toutes les informations nécessaires au bon fonctionnement de l'application.
- Enfin, la **communication via API REST** permet de relier l'interface utilisateur au serveur backend de façon standardisée. Cette architecture client-serveur repose sur des appels HTTP organisés par endpoints, facilitant la scalabilité, la maintenance, et même l'intégration future d'applications tierces. L'ensemble de ces compétences techniques constitue un socle essentiel pour tout développeur web full-stack moderne.
- Intégration d'une **API d'IA** dans une chaîne de traitement. Nous devons apprendre à concevoir un pipeline complet permettant d'analyser un document PDF ou DOCX, d'en extraire le contenu textuel, puis de le transmettre à une API d'intelligence artificielle. Celle-ci génère automatiquement un QCM pertinent à partir du contenu.



- Implémentation d'un système de suivi de performance utilisateur. Développé une fonctionnalité permettant d'enregistrer les scores des utilisateurs à chaque session de quiz, puis de les visualiser sous forme d'historiques et de graphiques.
- **Objectifs Pédagogiques :**
 - Approfondir l'usage de **JavaScript** : en développant des interfaces réactives et dynamiques, nous avons renforcé notre maîtrise du DOM, des événements, ainsi que des appels asynchrones pour le traitement des données.
 - Mettre en pratique une **gestion de projet** : nous avons travaillé en équipe avec des outils de suivi (GitHub, etc) et des sprints hebdomadaires permettant un avancement progressif, structuré et collaboratif.
 - Expérimenter un **cycle complet de développement** : de la conception jusqu'à la soutenance, nous avons couvert toutes les étapes clés : analyse des besoins, prototypage, implémentation, test, amélioration continue et documentation.

Ce projet nous permettra d'explorer des concepts avancés tels que la programmation événementielle et fonctionnelle, qui sont fondamentaux pour le développement d'applications modernes. En combinant ces approches, nous pourrons non seulement améliorer la réactivité de notre application, mais aussi garantir la clarté et la fiabilité de notre code. Ainsi, ce projet représente une opportunité unique de consolider nos compétences techniques tout en développant une application performante et innovante. Au cours de la réalisation de notre projet, nous allons approfondir nos compétences dans les domaines de la programmation événementielle et fonctionnelle. En intégrant des gestionnaires d'événements pour répondre aux interactions utilisateur, nous renforcerons notre capacité à créer des interfaces réactives et intuitives. Parallèlement, l'adoption de principes fonctionnels nous permettra de structurer notre code de manière plus claire et maintenable, en favorisant l'immuabilité et l'utilisation de fonctions pures. Cette double approche nous aidera à développer une application robuste, capable de gérer efficacement les événements tout en garantissant la fiabilité et la lisibilité du code. En combinant ces deux paradigmes, nous visons à améliorer non seulement la performance de notre projet, mais aussi la qualité globale de notre développement logiciel.

L'un des objectifs clés de ce projet, en termes de compétences techniques, est de maîtriser le Document Object Model (DOM) et de développer des compétences avancées en programmation asynchrone. La maîtrise du DOM est essentielle pour manipuler dynamiquement le contenu et la structure des pages web. En apprenant à interagir efficacement avec les éléments du DOM, nous pourrons créer des interfaces utilisateur interactives et réactives, capables de répondre instantanément aux actions des utilisateurs. Cette compétence nous permettra de mettre à jour le contenu des pages, de modifier les styles et de gérer les événements de manière fluide, améliorant ainsi l'expérience utilisateur globale.



Parallèlement, la programmation asynchrone est indispensable pour gérer les opérations qui prennent du temps, telles que les appels réseau ou l'accès aux bases de données, sans bloquer l'exécution du reste du code. En utilisant des techniques telles que les promesses, les fonctions `async/await` et les `callbacks`, nous pourrons écrire un code plus performant et réactif. Cette approche permet de gérer plusieurs tâches simultanément, optimisant ainsi les performances de notre application et garantissant une expérience utilisateur fluide et sans interruption.

L'utilisation de Node.js dans un projet nous permettra de développer une gamme complète de compétences essentielles pour le développement d'applications web modernes, tant au niveau du front-end que du back-end. En intégrant Node.js, nous pouvons créer un environnement d'exécution JavaScript côté serveur, ce qui facilite la gestion des requêtes asynchrones et l'interaction avec les bases de données. Cette approche nous permet de développer des API RESTful robustes et évolutives, capables de gérer efficacement les échanges de données entre le client et le serveur. De plus, Node.js favorise l'utilisation de modules et de bibliothèques partagées, améliorant ainsi la cohérence et la réutilisabilité du code entre le front-end et le back-end. En maîtrisant Node.js, nous renforçons notre capacité à créer des applications full-stack performantes, tout en optimisant les performances et la scalabilité de notre infrastructure. Cette compétence est particulièrement précieuse dans un contexte où l'intégration fluide entre les différentes couches de l'application est cruciale pour offrir une expérience utilisateur harmonieuse et réactive.

La mise en place d'une communication entre le front-end et le back-end via REST permet de développer des compétences cruciales pour le développement d'applications web modernes et interactives. En utilisant REST, nous apprenons à concevoir des API claires et bien structurées, qui facilitent l'échange de données entre le client et le serveur de manière standardisée et efficace. Cette approche nous permet de maîtriser les principes de l'architecture client-serveur, en assurant une séparation nette des préoccupations et une meilleure organisation du code. De plus, la création d'endpoints RESTful nous oblige à adopter des pratiques de développement rigoureuses, telles que la gestion des états HTTP, l'authentification et la sécurisation des données. Enfin, cette compétence favorise l'interopérabilité entre différents systèmes et services, permettant ainsi de créer des applications plus flexibles et évolutives, capables de s'intégrer facilement à d'autres technologies et plateformes.

En combinant ces compétences, nous serons en mesure de développer des applications web modernes et performantes, capables de répondre efficacement aux besoins des utilisateurs tout en offrant une expérience interactive et agréable. Ce projet représente donc une opportunité précieuse pour approfondir notre expertise technique et pour appliquer ces concepts de manière concrète et pratique.



Présentation de l’Idée du Projet

Knowledge Quest est une plateforme web conçue pour accompagner les étudiants dans leurs révisions en proposant une approche interactive et moderne de l’apprentissage. L’objectif principal de la plateforme est de faciliter la création de supports d’entraînement personnalisés, en permettant aux utilisateurs de générer automatiquement des questionnaires à choix multiples (QCM) à partir de leurs propres documents de cours. Les étudiants ont également la possibilité de concevoir manuellement leurs questionnaires, afin d’adapter parfaitement le contenu aux thématiques ou aux niveaux de difficulté souhaités.

Ce projet se distingue par son innovation majeure : l’intégration d’une API d’intelligence artificielle dédiée à l’analyse de documents pédagogiques. Grâce à cette technologie avancée, Knowledge Quest est capable de lire, comprendre et extraire les éléments clés d’un fichier, qu’il s’agisse de formats PDF ou DOCX. À partir de cette analyse, la plateforme génère automatiquement des questions pertinentes et cohérentes, en lien direct avec les contenus étudiés. Ce processus automatisé représente un gain de temps considérable pour les étudiants, qui peuvent ainsi se concentrer sur l’essentiel : l’apprentissage et la mémorisation.

En outre, l’utilisation de l’intelligence artificielle permet de proposer une expérience d’apprentissage beaucoup plus personnalisée. Chaque QCM généré est unique car il dépend directement du contenu fourni par l’utilisateur, ce qui garantit une adéquation optimale entre les exercices proposés et les connaissances réellement attendues dans le cadre académique. Cette personnalisation contribue à rendre les sessions de révision plus efficaces et engageantes, en ciblant précisément les besoins et les lacunes de chaque étudiant.

Fonctionnalités principales :

Knowledge Quest offre à ses utilisateurs la possibilité de **téléverser facilement des fichiers de cours au format PDF ou DOCX**. Cette fonctionnalité est essentielle puisqu’elle constitue la première étape du processus d’automatisation des révisions. En quelques clics, les étudiants peuvent importer leurs documents directement depuis l’interface web, sans avoir besoin de procéder à des opérations complexes de formatage ou de conversion.

Une fois le document chargé, l’utilisateur peut lancer la **génération automatique d’un questionnaire à choix multiples grâce à l’intégration d’une API d’intelligence artificielle**. Cette technologie est capable d’analyser le contenu textuel du fichier, d’en extraire les informations principales et de formuler des questions pertinentes et cohérentes. Le processus est totalement automatisé et permet de transformer un cours complet en un outil de révision structuré, en quelques secondes seulement.

En complément de cette automatisation, Knowledge Quest propose également une fonctionnalité de **création manuelle de questionnaires**. Cette option est destinée aux utilisateurs qui souhaitent conserver un contrôle total sur le contenu pédagogique ou qui préfèrent concevoir des exercices personnalisés adaptés à des besoins spécifiques. Grâce à



une interface simple et intuitive, il est possible d'ajouter manuellement des questions et des réponses, en configurant librement la difficulté, le domaine ou les thèmes abordés.

La plateforme permet ensuite aux utilisateurs de **participer à des sessions de quiz interactives** basées sur les questionnaires générés ou créés manuellement. Lors de ces sessions, chaque question est affichée de manière claire, avec un système de correction immédiate qui informe l'étudiant de la justesse de ses réponses. Cette approche dynamique contribue à renforcer la mémorisation des connaissances et à maintenir l'engagement de l'utilisateur tout au long de sa session d'entraînement.

Knowledge Quest intègre également un module de **suivi et d'analyse des résultats**. Chaque session de quiz est enregistrée dans l'historique de l'utilisateur, permettant ainsi de consulter des statistiques détaillées sur les performances : score moyen, taux de réussite, évolution dans le temps. Ces données sont présentées sous forme de graphiques et de tableaux de bord, offrant une visibilité claire sur les progrès accomplis et les axes d'amélioration.

L'interface utilisateur a été pensée pour être **fluide, réactive et entièrement responsive**. Que ce soit sur ordinateur, tablette ou smartphone, l'utilisateur bénéficie d'une expérience optimale, sans ralentissement ni problème d'affichage. L'ergonomie et l'intuitivité de la plateforme ont été au cœur des préoccupations dès les premières étapes de développement.

Enfin, l'authentification des utilisateurs est gérée de manière sécurisée, avec la possibilité de **se connecter via différentes API d'authentification** modernes. Ce système garantit la protection des données personnelles tout en offrant une grande flexibilité d'accès.

En s'appuyant sur les avancées technologiques en matière d'IA et en plaçant l'expérience utilisateur au centre de sa conception, Knowledge Quest apporte une réponse concrète aux défis de la révision moderne, tout en ouvrant la voie à de nombreuses évolutions futures dans le domaine de l'apprentissage numérique. Ce projet répond donc à des besoins concrets du monde étudiant et s'inscrit dans une démarche de digitalisation de l'apprentissage. Il pourrait également être étendu à d'autres domaines : formation professionnelle, préparation à des concours ou même permettre de délivrer des certificats dans le domaine informatique par exemple.



2. Analyse et Conception

Étude des Besoins

Les utilisateurs cibles de l'application Knowledge Quest sont principalement les étudiants de l'enseignement supérieur (licence, master, écoles d'ingénieurs) ayant besoin d'un outil de révision efficace et rapide. Les enseignants ou formateurs peuvent également utiliser la plateforme pour générer des exercices adaptés à leurs cours.

Les fonctionnalités attendues sont nombreuses : la possibilité de téléverser des documents (PDF, DOCX), de générer automatiquement des QCM à partir de ces contenus, de créer manuellement des questionnaires, de lancer des quiz interactifs avec correction immédiate, et enfin de suivre les performances à travers des statistiques détaillées.

Pour garantir la qualité de l'expérience utilisateur, l'application doit respecter certaines contraintes : une interface fluide et responsive, une communication sécurisée entre le frontend et le backend, une gestion rigoureuse des erreurs, ainsi qu'une base de données performante et adaptée à la structure des données (NoSQL).

Le cahier des charges du projet Knowledge Quest précise les besoins, les objectifs fonctionnels et techniques ainsi que les contraintes à respecter, comme illustré ci-dessous (À consulter en annexe).

CAHIER DES CHARGES - KNOWLEDGE QUEST					
	Fonctionnalité	Description	Outils techniques	Langages	Importance
3	Téléversement de fichiers (PDF/DOCX)	Permet aux utilisateurs de charger leurs documents de cours pour traitement.	API Upload Document, Node.js/file	JavaScript	5
4	Génération de QCM via IA	Extraction de contenu et création automatique de QCM depuis documents uploadés	API OPEN AI, Node.js	JavaScript	4
5	Création manuelle de QCM	Interface permettant à l'utilisateur de rédiger manuellement ses propres questions.	Programmation Web	JavaScript	5
6	Participation à des quiz	Affichage interactif des QCM générés avec réponses et correction.	DOM Manipulation	JavaScript	5
7	Suivi des performances	Enregistrement des résultats utilisateurs et affichage de statistiques.	MongoDB, Node.js / Chart.js	JavaScript, Node.js	4
8	Authentification Microsoft	Connexion via OAuth pour sécuriser et personnaliser l'accès.	OAuth2, Passport.js, Node.js/Azure AD	JavaScript, Node.js	2
9	Authentification Google	Connexion via OAuth pour sécuriser et personnaliser l'accès.	OAuth2, Passport.js, Node.js/Firebase	JavaScript, Node.js	2
10	Gestion des sessions utilisateurs	Maintien de la session utilisateur pour personnaliser l'expérience.	Node.js/Cookies	JavaScript, Node.js	4
11	Interface responsive	Affichage adapté aux écrans mobiles, tablettes et desktop.	CSS Flexbox/Grid, Media Queries	HTML, CSS, JavaScript	1
12	Interface utilisateur esthétique	Affichage statique esthétique	Visual Studio Code	CSS	5
13	Interface fluide	Affichage dynamique esthétique	Visual Studio Code	JavaScript	5
14	Arborescence claire et bien structurée	L'organisation des fichiers doit être irréprochable au vu du nombre de fichier.	Visual Studio Code	Fichier	5
15	Communication Back Front Sécurisée	La communication passe par un clé JWT.	Visual Studio Code, Node.js	Java JavaScript	5
16	Bouton de navigation	Des boutons interactifs doivent permettre de naviguer au travers des pages du site.	Visual Studio Code, Node.js	Office on the web Frame	4
17	Base de données	La base de donnée est en NoSQL.	Mongo DB Compass	JSON	5
18	Enregistrement des logs	Nous devons enregistrer les logs d'erreur notamment	Visual Studio Code	LOG	3
19	Notifications	L'application envoie des notifications de rappel.	DOM Manipulation	JavaScript	1
20	INDEX				
21	Bordereau fin de page	Le bordereau est composé de liens utiles et légaux.	Visual Studio Code	HTML	1
22	Accueil avec un bouton central	Ce bouton mis en avant sert à rediriger vers la création d'un compte	Visual Studio Code	JavaScript	5
23	Pages "Fonctionnalité"	Cette page présente les raisons de choisir Knowledge Quest ?	Visual Studio Code	HTML	4
24	Pages "Comment ça marche"	Cette page présente les processus d'utilisation de Knowledge Quest.	Visual Studio Code	HTML	4

Image 1 : extrait du cahier des charge



Modèle Conceptuel

Le modèle conceptuel du projet Knowledge Quest repose sur une organisation claire des interactions entre les utilisateurs, les services proposés par la plateforme, et les différentes couches techniques qui composent l'application. Cette modélisation permet d'avoir une vision globale et structurée du fonctionnement du système, depuis l'interface utilisateur jusqu'à la base de données. Le frontend est construit avec du JavaScript natif accompagné de HTML et CSS, tandis que le backend repose sur Node.js et Express.js. La base de données MongoDB permet de structurer les QCM et les résultats de manière flexible.

Afin d'illustrer le comportement attendu de l'application et les principales actions réalisables par un utilisateur, **un diagramme de cas d'utilisation** a été conçu. Ce schéma présente les différentes fonctionnalités accessibles, telles que la création d'un compte, l'authentification, l'upload de documents, la génération de QCM via l'intelligence artificielle, ainsi que la participation aux quiz et la consultation des résultats. Il permet de mettre en évidence la relation entre l'utilisateur et le système, en identifiant les cas d'utilisation majeurs qui structurent l'expérience sur la plateforme.

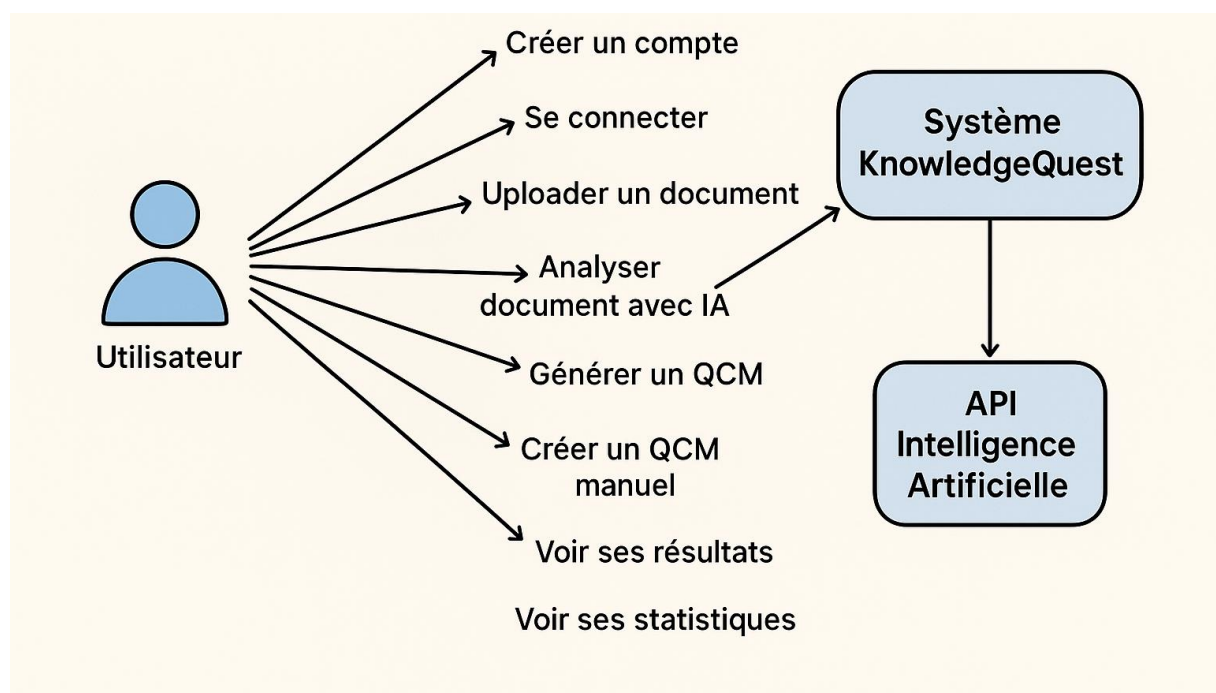


Image 2 : Diagramme de cas d'utilisation de Knowledge Quest.

Ce diagramme illustre les principales interactions possibles entre l'utilisateur et le système, telles que la création d'un compte, l'upload de documents, la génération de QCM, la participation à un test, et la consultation des résultats.



Dans un second temps, un **diagramme d'architecture** a été élaboré afin de modéliser l'organisation technique du projet. Knowledge Quest suit une architecture web classique en trois couches : le Frontend, le Backend, et la Base de données. Le Frontend, composé de pages HTML, CSS et JavaScript, interagit avec un service API Client pour envoyer et recevoir les données. Ce service communique avec le Backend, développé en Node.js avec Express, qui gère la logique métier via différents contrôleurs (sessions, QCM, authentification) et se connecte également à une API externe d'intelligence artificielle pour l'analyse des documents. La persistance des données est assurée par une base MongoDB, adaptée à la flexibilité requise par les structures dynamiques de QCM et de sessions utilisateur.

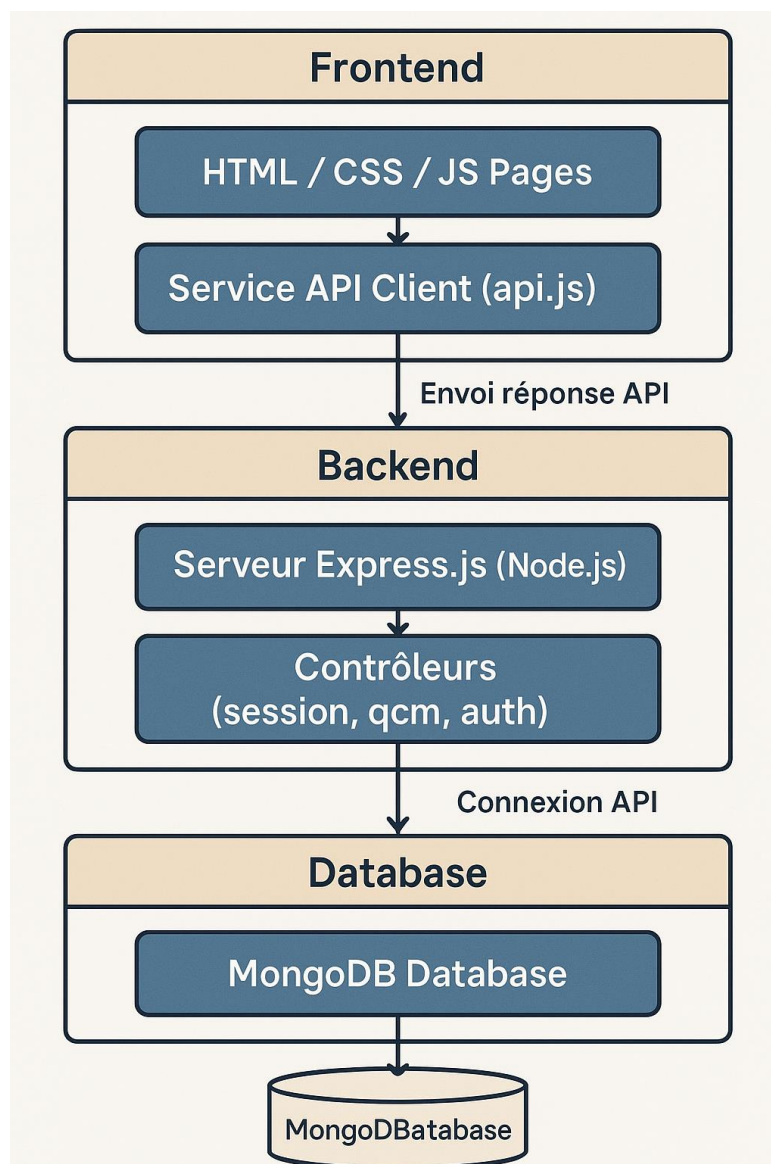


Image 3 : Diagramme d'architecture de Knowledge Quest.

Ce schéma présente l'organisation technique de l'application, structurée en trois couches : Frontend, Backend, et Base de données, avec la communication via des APIs REST et l'intégration d'une API d'intelligence artificielle externe.



Ces deux diagrammes offrent ainsi une compréhension synthétique et complémentaire du projet : l'un du point de vue fonctionnel (cas d'utilisation) et l'autre du point de vue technique (architecture logicielle).

Justification des Choix Techniques

Le langage JavaScript a été choisi pour sa grande polyvalence, permettant de développer aussi bien le côté client que le côté serveur, notamment grâce à l'environnement d'exécution Node.js. Express.js a été retenu pour sa légèreté, sa simplicité de mise en œuvre et son efficacité dans la création d'API RESTful, indispensables à la communication entre les différentes couches de l'application. Concernant la base de données, MongoDB s'est imposé comme le choix idéal grâce à sa structure flexible de type NoSQL, parfaitement adaptée aux objets dynamiques tels que les QCM, les utilisateurs et les historiques de résultats.

Pour la gestion de projet, plusieurs outils complémentaires ont été utilisés afin de garantir un suivi efficace et une bonne organisation du travail. GitHub a permis d'assurer un suivi rigoureux du versioning et de faciliter la collaboration entre les membres de l'équipe grâce à une gestion centralisée du code source. Teams a été utilisé pour organiser les tâches, planifier les différentes étapes du projet et assurer une meilleure visualisation de l'avancement ainsi que la priorisation des objectifs. La communication quotidienne a été assurée via Discord, garantissant ainsi une coordination fluide et réactive entre les membres de l'équipe.

Concernant l'environnement de développement, Visual Studio Code a été notre outil principal, choisi pour sa légèreté, ses nombreuses extensions, et sa compatibilité avec notre stack technique. Pour tester toutes nos API REST, nous avons utilisé Postman, un outil indispensable pour simuler les différentes requêtes et vérifier la conformité et la robustesse des réponses du serveur. Le stockage des données a été confié à MongoDB, une base de données NoSQL adaptée à notre besoin de flexibilité pour stocker les utilisateurs, les QCM et les résultats. Enfin, Firebase a été utilisé de manière spécifique pour gérer uniquement l'authentification des utilisateurs connectés via Google, offrant ainsi une intégration simple et sécurisée du login avec un compte Google.

Cette combinaison d'outils techniques et de plateformes collaboratives nous a permis de développer Knowledge Quest de manière structurée, rapide et efficace, tout en assurant la qualité et la fiabilité des différentes briques du projet.

De plus, afin de structurer le déroulement du projet et d'assurer une progression méthodique, nous avons élaboré un **Diagramme de Gantt**. Cet outil nous a permis de visualiser l'ensemble des phases du projet, de planifier les différentes tâches à réaliser, et de fixer des échéances claires pour chaque étape majeure.

Le Diagramme de Gantt est un outil de planification particulièrement utile dans la gestion de projet. Il permet de représenter visuellement la répartition des tâches dans le



temps, de définir les dépendances entre elles, et de suivre l'avancement global. Grâce à cette représentation graphique, chaque membre de l'équipe pouvait anticiper les étapes suivantes, évaluer sa charge de travail, et ajuster son rythme en fonction des délais impartis.

Concrètement, notre projet a été divisé en plusieurs grands sprints répartis sur huit semaines. Les premières semaines ont été consacrées à la planification et à la conception du projet, suivies du développement du frontend et du backend en parallèle. L'intégration de l'API d'intelligence artificielle ainsi que la mise en place de la base de données ont été réalisées en milieu de parcours. Les dernières semaines ont été réservées au débogage, aux tests fonctionnels et à la validation finale du projet.

Le Diagramme de Gantt présenté ci-dessous illustre cette organisation temporelle et la répartition des tâches au sein de notre équipe.

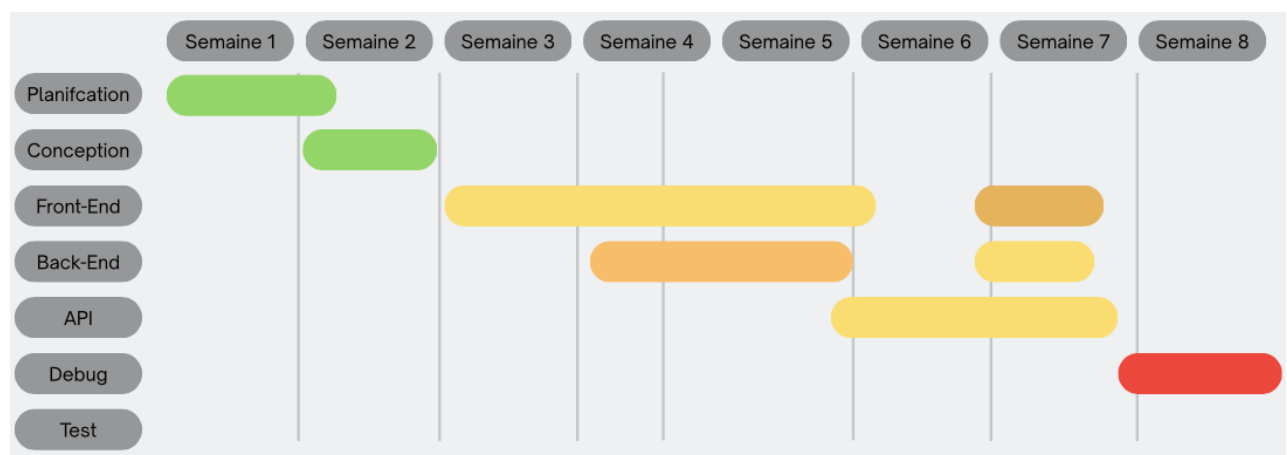


Image 4 : Diagramme de Gantt de Knowledge Quest.

Ce diagramme représente la planification du projet sur huit semaines, avec la répartition des tâches principales et les différentes échéances fixées.

Après avoir défini les choix techniques, organisé la planification du projet et réparti les différentes phases de développement, nous avons entamé la réalisation concrète de la plateforme Knowledge Quest. La phase de développement et d'implémentation a été conduite de manière progressive, en suivant l'organisation établie et en respectant les objectifs définis pour chaque sprint. Dans la section suivante, nous détaillerons le processus de développement, la méthodologie adoptée ainsi que les outils spécifiques utilisés pour construire l'application.



3. Développement et Implémentation

Description du Développement

Le développement du projet Knowledge Quest s'est articulé autour de plusieurs étapes clés, respectant une approche méthodique et progressive. Nous avons commencé par l'initialisation de l'environnement de travail, avec la création des structures de dossiers pour le frontend et le backend, ainsi que la configuration du dépôt GitHub pour assurer un suivi rigoureux du versioning et de la collaboration entre les membres de l'équipe.

Dans un second temps, l'accent a été mis sur le développement du backend. Un serveur a été mis en place à l'aide de Node.js et du framework Express.js. Les routes API ont été créées pour permettre la gestion de l'upload de documents, l'interaction avec l'API d'intelligence artificielle externe, et la sauvegarde des QCM dans notre base de données MongoDB. À cette étape, une attention particulière a été portée à la gestion des erreurs ainsi qu'à la validation des données entrantes pour garantir la robustesse et la fiabilité du système.

La communication entre le frontend et le backend repose sur une architecture REST API. Cette approche a permis de standardiser les échanges entre les différentes couches du projet, facilitant ainsi la modularité, l'évolutivité et le test des différentes fonctionnalités. Un schéma simplifié de cette communication est présenté ci-dessous.

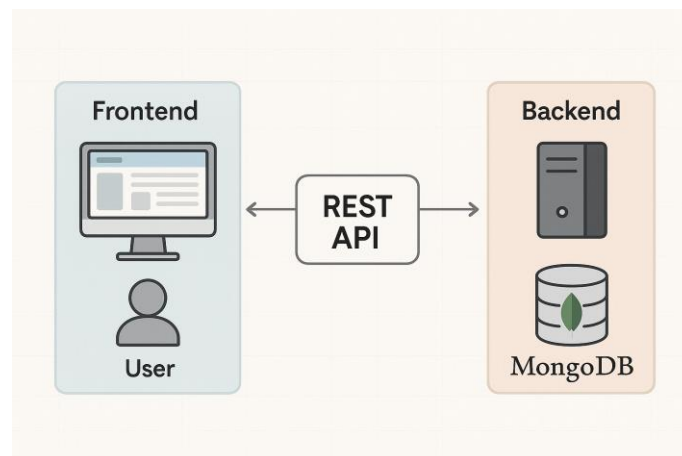


Image 5 : Communication via REST API.

Ce schéma illustre l'échange d'informations entre l'interface utilisateur, le serveur et la base de données à travers une API REST standardisée.

Le développement du frontend s'est déroulé en parallèle. Nous avons conçu des interfaces en HTML et CSS, enrichies par des scripts JavaScript pour dynamiser l'expérience utilisateur. Le frontend prend en charge l'envoi des documents vers le backend, la réception et l'affichage des QCM générés, ainsi que l'interaction avec les utilisateurs durant les sessions de



quiz. Des composants dynamiques ont été développés pour la gestion des formulaires, l'affichage des questions/réponses, et la visualisation des résultats sous forme de statistiques.

Enfin, un système complet de suivi de performance des utilisateurs a été implémenté. À chaque session de quiz, les résultats sont enregistrés, les scores sont calculés, et les performances sont synthétisées dans des tableaux de bord accessibles à l'utilisateur. Ces statistiques permettent aux étudiants de suivre leur évolution et d'adapter leurs stratégies de révision en fonction de leurs résultats précédents.

Méthodes et Outils Utilisés

Pour mener à bien le développement de Knowledge Quest, nous avons mobilisé plusieurs outils essentiels, centrés autour du développement collaboratif et du contrôle de version. Visual Studio Code a été notre environnement de développement principal. Son ergonomie, sa légèreté et sa compatibilité avec notre stack technologique nous ont permis d'écrire, organiser et maintenir le code efficacement tout au long du projet.

Le suivi des versions et la collaboration entre les membres de l'équipe ont été assurés grâce à Git et GitHub. Nous avons adopté une organisation structurée en branches distinctes pour le développement : chaque membre travaillait sur sa propre branche dédiée, notamment pour le développement des différentes parties de l'API backend. Cette approche nous a permis de travailler en parallèle sans générer de conflits majeurs. Une fois les développements terminés, les branches individuelles étaient fusionnées sur la branche principale (main) après relecture et validation, garantissant ainsi l'intégration progressive et sécurisée du projet final.

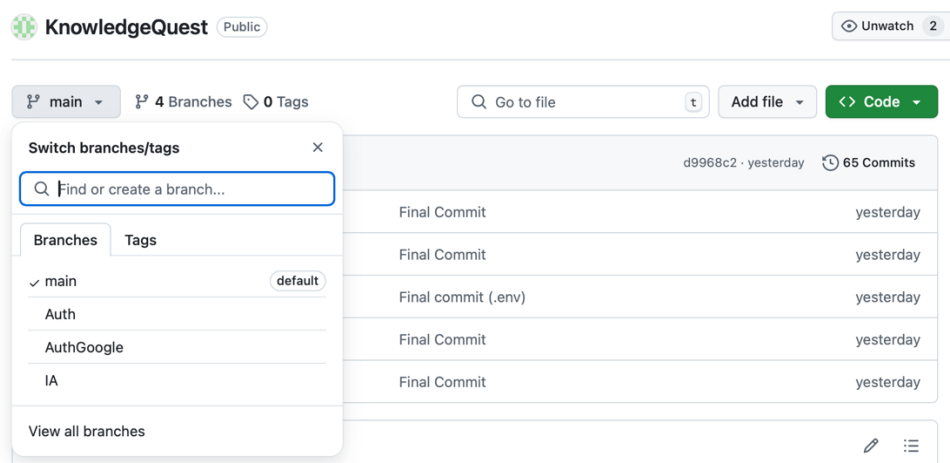


Image : Espace de travail GitHub



La communication continue et la coordination de l'équipe ont été facilitées par l'utilisation de Discord, où nous échangeons régulièrement sur l'état d'avancement, les points techniques à résoudre, et la planification des différentes étapes du projet.

Pour la phase de tests, Postman a été utilisé comme outil principal afin de simuler les différentes requêtes HTTP (GET, POST, PUT, DELETE) et vérifier le bon fonctionnement de toutes les routes API que nous avons développées. Cet outil a été indispensable pour s'assurer de la validité des échanges entre le frontend et le backend avant l'intégration complète.

Enfin, MongoDB a été utilisé pour stocker l'ensemble des données relatives aux utilisateurs, aux QCM et aux sessions de quiz. Firebase a été utilisé de manière spécifique uniquement pour gérer l'authentification via Google, permettant aux utilisateurs de se connecter facilement et de manière sécurisée.

Cette organisation, combinant GitHub, Visual Studio Code, Discord et Postman, nous a permis de travailler efficacement en équipe, de maintenir un haut niveau de qualité dans notre code, et de garantir la stabilité progressive de l'application tout au long de son développement.



Décomposition du Projet

Pour garantir une organisation efficace et une progression fluide du développement, nous avons procédé à une décomposition du projet en modules fonctionnels distincts. Cette segmentation nous a permis de mieux répartir les responsabilités et d'assurer une complémentarité entre les membres de l'équipe.

Frontend

Frontend de l'application :

Le frontend de l'application a été entièrement conçu en HTML, CSS et JavaScript afin de proposer une interface utilisateur fluide et interactive. Chaque page a été pensée pour offrir une navigation intuitive, un accès rapide aux fonctionnalités principales et une expérience utilisateur agréable. Le JavaScript est au cœur des interactions : il gère le chargement de fichiers, la navigation dynamique entre les pages, l'affichage et le passage des QCM, ainsi que la gestion des réponses des utilisateurs. Concernant le suivi des performances, des modules de statistiques et de tableaux de bord permettent de calculer les scores, d'afficher la progression des utilisateurs et de centraliser les données liées à leurs résultats, offrant une visualisation claire et détaillée.

Page Index :

La page d'accueil de Knowledge Quest joue un rôle essentiel en tant que premier point de contact avec l'utilisateur. Dès l'ouverture de la page, un script JavaScript s'exécute pour vérifier automatiquement si l'utilisateur est déjà authentifié. Cette vérification se fait en consultant le localStorage, où est stocké un éventuel token d'authentification. Si un token est présent, l'utilisateur est immédiatement redirigé vers son tableau de bord sans passer par la page d'accueil, ce qui améliore considérablement l'expérience en évitant des étapes inutiles. En revanche, si aucun token n'est détecté, les boutons permettant de se connecter ou de créer un compte sont affichés dynamiquement afin de guider l'utilisateur vers les bonnes options.

Le JavaScript de cette page sert également à initialiser l'ensemble de l'application côté client. À travers un fichier d'entrée principal, plusieurs modules sont importés et configurés : notamment l'authentification, la gestion des appels API, ainsi que l'application des styles personnalisés. Une attention particulière est portée pour éviter que cette initialisation ne se fasse plusieurs fois grâce à l'utilisation d'une variable de contrôle. De plus, les services essentiels comme l'authentification et l'accès aux API sont exposés au niveau global (window) afin qu'ils puissent être utilisés facilement par d'autres scripts si nécessaire.

La gestion des erreurs est elle aussi intégrée au frontend via un module spécifique. Lorsqu'une erreur survient, par exemple lors d'une requête API ou d'une interaction



utilisateur incorrecte, une fonction dédiée permet d'afficher un message d'erreur de manière élégante. Si un élément spécifique est prévu pour afficher les erreurs, le message y est injecté et mis en forme automatiquement. Sinon, une notification flottante de type "toast" est générée dynamiquement et disparaît après quelques secondes, rendant la gestion des erreurs beaucoup plus visible et intuitive pour l'utilisateur.

Un autre aspect important de l'interactivité de la page concerne la gestion des modales. Un module JavaScript spécialisé s'occupe d'initialiser les modales présentes sur la page, comme celles pour l'A-propos, le Contact et la Politique de confidentialité. Lorsqu'un utilisateur clique sur un lien déclencheur, la modale correspondante s'affiche avec une animation d'ouverture douce. Les modales peuvent être fermées en cliquant sur une croix, en cliquant à l'extérieur ou en appuyant sur la touche Échap. Cette approche assure une utilisation confortable et accessible, tout en évitant de perturber la navigation sur la page.

Dans l'ensemble, la page index combine plusieurs scripts pour garantir une navigation fluide, une personnalisation avancée et une gestion robuste des erreurs et des interactions utilisateur. Toute cette logique JavaScript est pensée pour rendre l'application agréable et réactive dès les premiers instants d'utilisation.

Style de l'application :

Le fichier `style.js` est dédié à la gestion dynamique des préférences visuelles de l'application. Son objectif principal est d'assurer l'application automatique des paramètres utilisateur enregistrés dans le `localStorage` et de garantir une mise à jour en temps réel de l'interface sans rechargement de page.

À l'initialisation via la fonction `initStyles()`, le script commence par vérifier si les styles ont déjà été appliqués grâce à un flag interne (`stylesApplied`). Cette précaution permet d'éviter toute réinitialisation ou application multiple non désirée des préférences. Si ce n'est pas le cas, `applySavedSettings()` est appelée pour récupérer et appliquer les préférences sauvegardées dans le `localStorage`, en les passant à la fonction `applyStylePreferences()`.

La fonction `applyStylePreferences(settings)` gère trois aspects principaux : l'activation du mode sombre (`darkMode`), le réglage de la taille de police (`fontSize` via un attribut `data-font-size` sur la racine du document), et l'application d'un thème de couleurs (`theme`). Si aucune préférence de thème n'est spécifiée, `applyDomainTheme()` est invoquée pour appliquer automatiquement un thème basé sur le domaine d'étude de l'utilisateur, en se basant sur les informations contenues dans `authData`.

Le script utilise également `setupStyleEventListeners()` pour ajouter des écouteurs sur les événements `settings-updated` et `storage`. Cela permet de réagir en temps réel à toute modification des préférences locales ou partagées entre onglets. Lorsqu'un changement est détecté, `handleSettingsUpdate(e)` ou `handleStorageChange(e)` sont déclenchées pour recharger et appliquer les nouvelles préférences instantanément.

Enfin, pour appliquer ou changer de thème, la fonction `setColorTheme(theme)` s'assure de retirer les anciennes classes CSS (`medicine-theme`, `law-theme`, `neutral-theme`) du `<body>` avant d'ajouter la nouvelle, garantissant ainsi une transition propre entre les thèmes.



Page Register :

La page d'inscription de Knowledge Quest permet aux nouveaux utilisateurs de créer un compte en remplissant un formulaire dynamique entièrement géré en JavaScript. Lors du chargement de la page, un contrôle est effectué pour vérifier si l'utilisateur est déjà authentifié en consultant le localStorage. Si un token est détecté, l'utilisateur est automatiquement redirigé vers le tableau de bord, garantissant une navigation rapide et cohérente.

Le processus d'inscription repose sur l'écoute d'un événement sur le bouton de validation du formulaire. Lors du clic, les données saisies par l'utilisateur (prénom, nom, email, mot de passe, confirmation du mot de passe et domaine d'étude) sont récupérées, validées localement et envoyées en POST à l'API backend. Des vérifications sont effectuées côté frontend pour s'assurer que tous les champs sont remplis, que les mots de passe correspondent et que les conditions d'utilisation sont acceptées avant d'envoyer la requête. Si une erreur est détectée durant cette phase de validation, un système de notification visuelle est déclenché via le module notification.js, qui affiche des messages d'erreur élégants et non intrusifs à l'utilisateur.

Si l'inscription réussit, un token d'authentification et les informations de l'utilisateur sont stockés dans le localStorage et une redirection vers le tableau de bord est automatiquement initiée après une courte notification de succès. En cas d'échec, que ce soit lié à un problème de saisie ou à une erreur serveur, un message d'erreur est affiché pour informer l'utilisateur en temps réel.

La page propose également deux méthodes alternatives d'inscription via Google et Microsoft. L'inscription avec Google utilise Firebase pour ouvrir une popup d'authentification OAuth, récupérer un token utilisateur, puis envoyer ce token au serveur pour création ou connexion du compte. L'inscription via Microsoft repose sur une simple redirection vers l'API backend avec un paramètre d'état permettant d'associer le domaine d'étude choisi avant l'authentification.

La gestion des notifications visuelles est assurée par le module notification.js, qui crée dynamiquement des alertes flottantes pour informer l'utilisateur de l'état de ses actions (succès, erreurs, avertissements, informations). Ces notifications apparaissent en haut de l'écran avec un style adapté selon le type de message et disparaissent automatiquement après quelques secondes ou peuvent être fermées manuellement.

Grâce à cette architecture JavaScript, l'expérience utilisateur est à la fois fluide, réactive et intuitive, assurant une inscription rapide et sans friction, tout en maintenant un retour immédiat en cas d'erreur ou de succès.

Page Login :

La page de connexion de Knowledge Quest permet aux utilisateurs existants de se connecter rapidement à leur compte. Dès le chargement de la page, un contrôle est effectué pour vérifier si un token est déjà présent dans le localStorage. Si un token valide est trouvé, l'utilisateur est automatiquement redirigé vers son tableau de bord sans avoir



à repasser par le formulaire de connexion, ce qui optimise l'expérience utilisateur en supprimant les étapes inutiles.

Le formulaire de connexion est entièrement géré en JavaScript. Lorsqu'un utilisateur clique sur le bouton de connexion, le script récupère l'email et le mot de passe saisis, vérifie que les deux champs sont remplis, puis envoie une requête POST au serveur pour tenter l'authentification. Si la connexion est réussie et qu'un token est renvoyé, celui-ci est sauvegardé dans le localStorage en même temps que les informations utilisateur, et l'utilisateur est redirigé vers son tableau de bord après un message de succès affiché à l'écran. En cas d'erreur, qu'elle soit liée à une mauvaise saisie, à un problème d'authentification ou à une erreur serveur, un message d'erreur est affiché de manière visible grâce à une fonction de notification personnalisée intégrée à la page.

Un bouton pour l'authentification via Google est également présent, mais la logique associée n'est pas encore entièrement implémentée dans ce fichier. Lors du clic sur ce bouton, une notification informative est simplement affichée pour prévenir l'utilisateur que cette fonctionnalité sera disponible prochainement.

Toute l'interaction de la page repose donc sur des traitements JavaScript côté client pour assurer une expérience rapide, dynamique et sans rechargement inutile, tout en garantissant une gestion propre des erreurs et des statuts utilisateur.

Page Dashboard :

La page de tableau de bord de Knowledge Quest est entièrement dynamique et personnalisée grâce au JavaScript. Lors du chargement de la page, les préférences utilisateur sauvegardées (comme le mode sombre, la taille de police ou le thème choisi) sont directement appliquées depuis le localStorage pour garantir une expérience cohérente avant même l'affichage du contenu. Un script principal (index.js) initialise l'application en important les modules d'authentification, de gestion API, et de style, en veillant à éviter toute double initialisation.

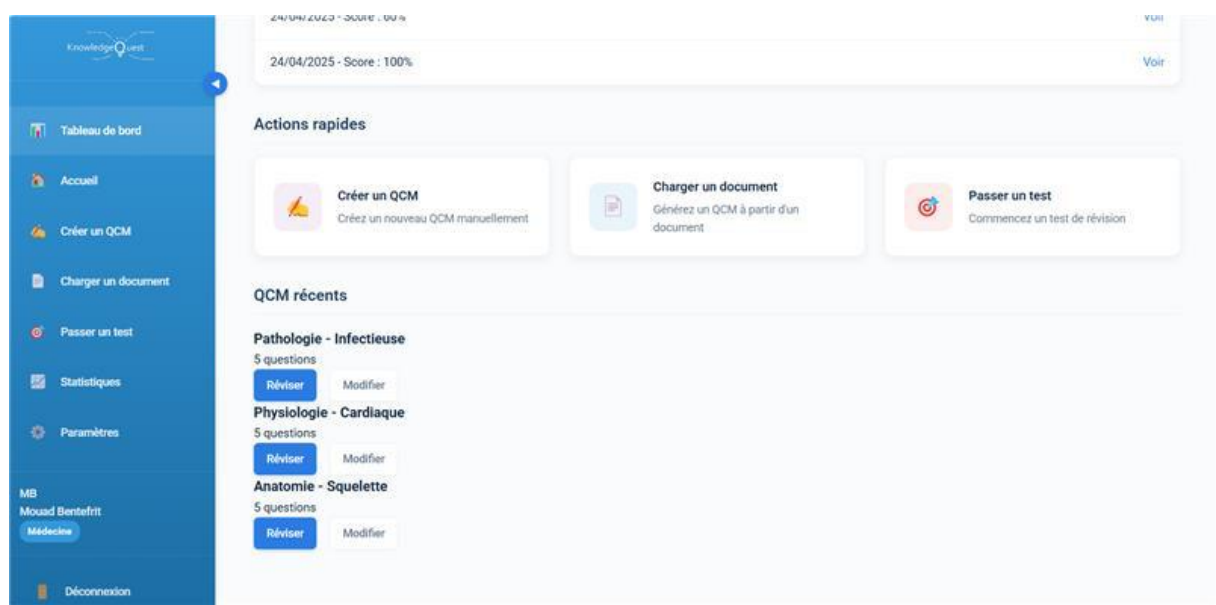
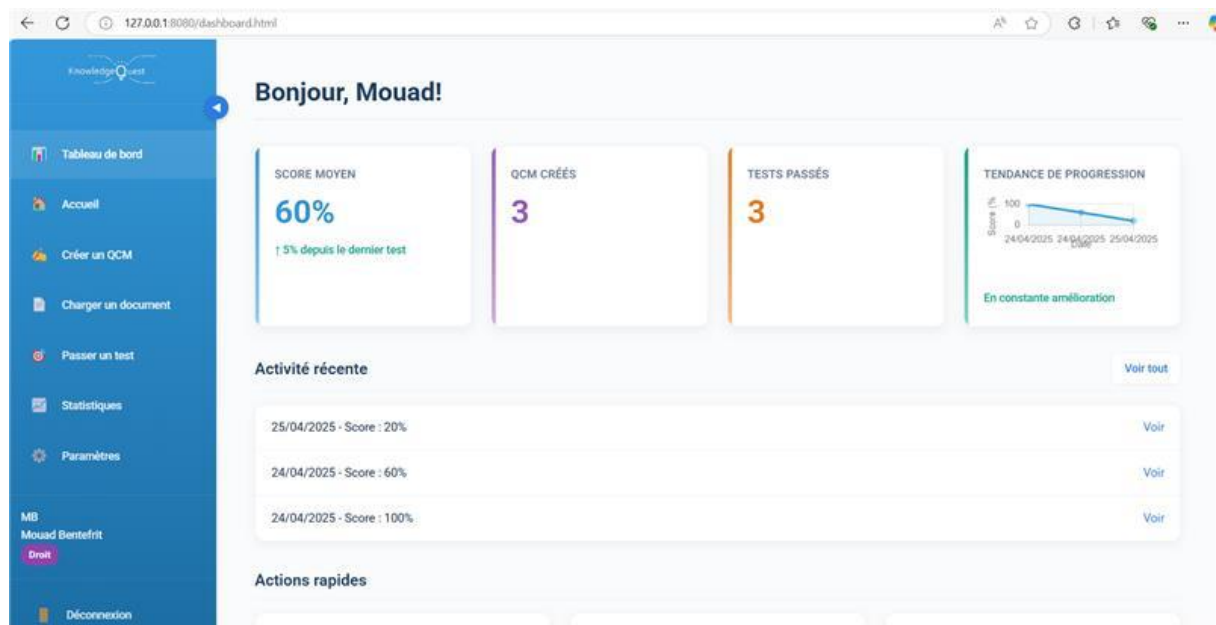
Le module dashboard-page.js prend ensuite le relais pour gérer la logique spécifique du tableau de bord. Lors de l'initialisation, il vérifie que l'utilisateur est bien connecté ; sinon, il redirige immédiatement vers la page de connexion pour des raisons de sécurité. Si un token est présent dans l'URL (suite à une authentification externe comme Google ou Microsoft), il est extrait, stocké, puis supprimé de l'URL pour éviter toute fuite accidentelle. Une fois l'utilisateur validé, ses informations personnelles (nom, domaine d'étude) sont affichées dynamiquement dans la barre latérale ainsi que dans le titre principal de la page.

Le tableau de bord affiche plusieurs statistiques en temps réel : score moyen, nombre de QCM créés, nombre de tests passés et évolution des performances sous forme de graphique généré avec Chart.js. Les scores récents et les QCM récents sont également affichés de manière dynamique en récupérant les données via les services qcmService et statsService, en s'assurant que seules les informations propres à l'utilisateur connecté sont présentées.



Un module complémentaire sidebar.js gère les interactions spécifiques à la barre latérale, notamment le repli/affichage de celle-ci avec animation, l'affichage dynamique du nom et du domaine utilisateur, ainsi que la déconnexion sécurisée qui vide le localStorage et redirige vers la page de connexion.

Tout l'ensemble fonctionne grâce à une structure modulaire et organisée, avec des appels asynchrones, de la gestion d'erreurs en temps réel via notifications, et une attention portée sur la performance et la fluidité pour offrir une expérience utilisateur optimale dès l'arrivée sur le tableau de bord.





Page Home :

La page d'accueil de Knowledge Quest propose une expérience personnalisée dès l'arrivée de l'utilisateur. Lors du chargement, un contrôle d'authentification est réalisé via le module auth, et si l'utilisateur n'est pas connecté, il est automatiquement redirigé vers la page de connexion pour garantir la sécurité de l'application. Si l'utilisateur est bien authentifié, son prénom est extrait et affiché dynamiquement dans le message de bienvenue pour renforcer la personnalisation de l'interface.

Le JavaScript de la page, via le fichier home-page.js, interagit ensuite avec le service statsService pour récupérer les statistiques de l'utilisateur, notamment son score moyen et son historique de tests passés. Le score moyen est affiché à l'intérieur d'un cercle de progression interactif ; l'utilisateur peut cliquer sur cet élément pour être redirigé directement vers la page de statistiques détaillées. Ce cercle est mis à jour dynamiquement avec le score réel récupéré depuis l'API.

La section d'activité récente est également générée de manière dynamique. En récupérant les dernières sessions de test de l'utilisateur, le script crée automatiquement des éléments dans la timeline, présentant les titres des QCM passés, les scores obtenus et les dates de passation. Chaque activité possède un lien rapide permettant de consulter les résultats détaillés du test correspondant. Si aucune activité n'est disponible, un message adapté est affiché pour indiquer l'absence de données.

En complément, la navigation latérale est entièrement dynamique grâce au module sidebar.js, qui permet notamment de gérer le repli de la barre et l'affichage du nom et du domaine d'étude de l'utilisateur. En cas d'erreur lors du chargement des statistiques ou de l'activité, une notification d'erreur est affichée grâce au système de gestion d'erreurs pour maintenir une communication claire avec l'utilisateur sans interrompre sa navigation.



L'ensemble de la page est conçu pour offrir à l'utilisateur une vue d'ensemble rapide de sa progression et lui proposer des actions immédiates adaptées à ses besoins du moment, dans une interface fluide, intuitive et 100% pilotée par JavaScript.



Page Créer un QCM :

La page "Créer un QCM" de Knowledge Quest permet aux utilisateurs de concevoir manuellement leurs propres questionnaires personnalisés. Dès l'ouverture de la page, le JavaScript vérifie que l'utilisateur est authentifié, sinon il est automatiquement redirigé vers la page de connexion. Le module create-qcm-page.js gère toute la logique d'interaction, en s'assurant que l'interface est initialisée une seule fois grâce à un indicateur de contrôle.

Le chargement des matières disponibles est effectué dynamiquement à partir du backend, en fonction du domaine d'étude de l'utilisateur (Médecine ou Droit). Une fois une matière sélectionnée, les thèmes associés sont automatiquement proposés dans un second menu déroulant. L'utilisateur a aussi la possibilité de créer une nouvelle matière directement depuis la page via un mini-formulaire dédié ; celle-ci sera enregistrée dans la base de données et ajoutée immédiatement aux options disponibles.

Le cœur de la création de QCM repose sur un formulaire dynamique où l'utilisateur peut ajouter manuellement autant de questions qu'il le souhaite. Chaque question comprend un énoncé, quatre choix de réponses, et une sélection obligatoire de la bonne réponse. Lors de l'ajout ou la suppression de questions, les numéros et l'indexation sont automatiquement recalculés pour garantir une cohérence visuelle et fonctionnelle. En validant le formulaire, toutes les données du QCM sont regroupées dans un objet et envoyées via l'API pour être enregistrées en base de données. Si l'utilisateur modifie un



QCM existant (détecté grâce aux paramètres d'URL), les champs sont préremplis automatiquement avec les données du QCM à modifier.

Des modèles de QCM sont également proposés sous forme de cartes cliquables pour accélérer la création, en important directement un ensemble de questions prédéfinies. Toute erreur durant la création, la sauvegarde ou le chargement des données est interceptée et communiquée via le système de notifications visuelles pour assurer une expérience fluide et informative à chaque étape.

The screenshot shows the 'Créer un QCM' (Create a QCM) form in the KnowledgeQuest application. The left sidebar contains navigation links: Tableau de bord, Accueil, Créer un QCM (highlighted), Charger un document, Passer un test, Statistiques, Paramètres, MB, Mouad Bentefrit, Médecine, and Déconnexion. The main form is titled 'Créer un QCM' and 'Informations générales'. It includes fields for 'Titre du QCM' (with an example 'Ex: Physiologie Cardiaque'), 'Matière' (a dropdown menu showing 'Anatomie'), and 'Thème' (a dropdown menu with the text 'Veuillez d'abord choisir une matière'). Below these fields is a section 'Ajouter une nouvelle matière' with a text input for 'Nom de la matière (ex: Anatomie)' and a 'Ajouter' button.

The screenshot shows the 'Questions' form in the KnowledgeQuest application. The left sidebar is identical to the previous screenshot. The main form is titled 'Questions' and has a '+ Ajouter une question' button in the top right corner. The form contains a section for 'Question 1' with a 'Supprimer' button. Below this is a large text area for the 'Énoncé' (Statement). At the bottom, there are four input fields for 'Réponse 1', 'Réponse 2', 'Réponse 3', and 'Réponse 4', each preceded by a radio button.



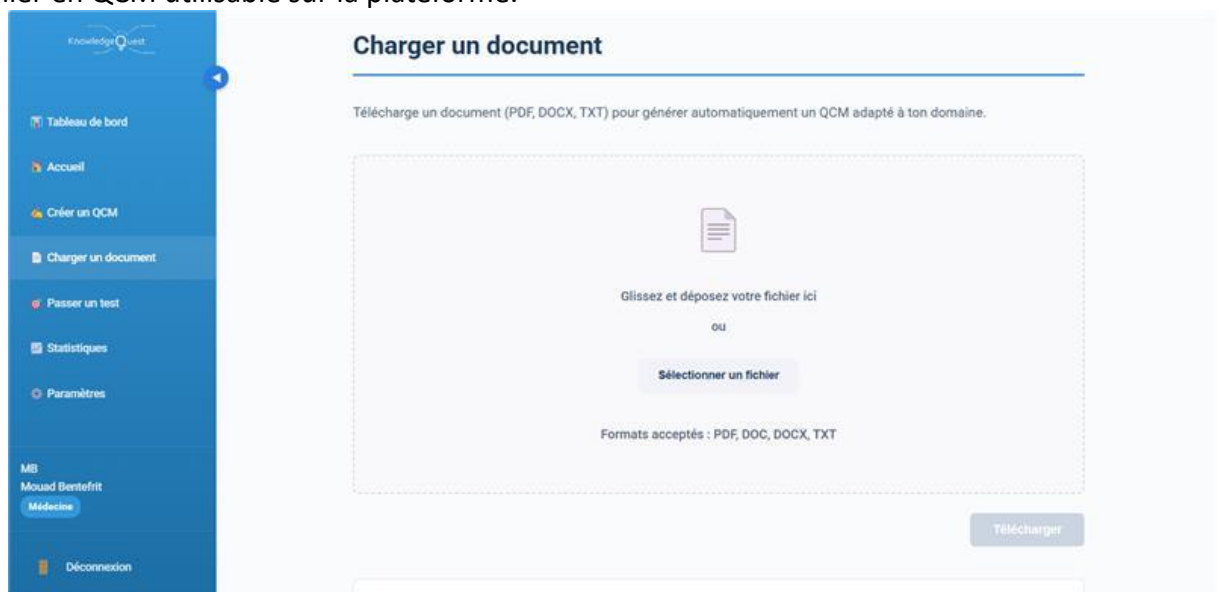
Page Charger un document :

La page "Charger un document" de Knowledge Quest permet aux utilisateurs d'envoyer un fichier texte (PDF, DOCX, TXT) pour générer automatiquement un QCM adapté à leur domaine d'étude. Dès l'ouverture de la page, le JavaScript vérifie l'authentification de l'utilisateur ; en l'absence de connexion, il est redirigé vers la page de login. Le script upload-page.js prend en charge toute la gestion du formulaire d'upload et de la génération de QCM.

Le système de dépôt de fichier repose sur une zone de drag-and-drop entièrement gérée en JavaScript. L'utilisateur peut soit glisser un fichier dans la zone prévue, soit cliquer pour en sélectionner un depuis son ordinateur. Des vérifications immédiates sont effectuées pour s'assurer que le fichier respecte les formats acceptés (.pdf, .doc, .docx, .txt) et qu'il ne dépasse pas une taille limite (10 Mo). Lorsqu'un fichier valide est chargé, son nom et sa taille sont affichés, et le bouton de soumission devient actif.

Lors de l'envoi du fichier, une barre de progression apparaît pour indiquer l'état de l'upload en temps réel. Si le téléchargement réussit, l'utilisateur peut choisir entre deux options : générer automatiquement un QCM via un appel API à l'IA, ou créer manuellement un QCM basé sur le document uploadé. Si le service d'IA rencontre un problème (comme un quota OpenAI dépassé ou une indisponibilité serveur), une solution alternative est immédiatement proposée pour continuer à travailler sans perte de données.

Le script gère également tous les retours d'erreurs de manière propre et visuelle grâce aux notifications utilisateur, garantissant ainsi que l'utilisateur est toujours informé de ce qu'il se passe, sans blocage de l'interface. Grâce à cette structure entièrement dynamique et interactive, la page offre une expérience fluide pour transformer un simple fichier en QCM utilisable sur la plateforme.





Page Passer un test :

La page "Passer un test" de Knowledge Quest est conçue pour permettre aux utilisateurs de lancer un QCM, répondre aux questions, suivre leur progression et enregistrer leurs résultats à la fin. Dès l'ouverture, le JavaScript s'assure que l'utilisateur est connecté ; s'il ne l'est pas, il est immédiatement redirigé vers la page de connexion pour sécuriser l'accès.

Si l'URL contient un identifiant de QCM (qcmId), le script charge directement le questionnaire correspondant en récupérant ses questions via qcmService. Sinon, la page affiche une liste interactive de tous les QCM disponibles que l'utilisateur peut sélectionner pour commencer un test. L'interface de test est entièrement dynamique : chaque question est affichée une par une, avec des boutons pour naviguer entre "Précédent", "Suivant" et "Terminer". Un chronomètre visible en haut de page indique le temps écoulé depuis le début du test.

Chaque réponse sélectionnée est stockée dans un tableau userAnswers, et la progression est automatiquement sauvegardée dans le localStorage, permettant de reprendre un test en cours même après une déconnexion ou un rechargement de la page. Avant de terminer, si des questions n'ont pas été répondues, une alerte de confirmation est affichée pour éviter toute soumission accidentelle.

Une fois le test terminé, les résultats sont compilés : chaque réponse est comparée à la bonne réponse pour calculer un score final en pourcentage. Le détail des réponses et le score sont ensuite envoyés au backend via sessionService pour être enregistrés. Si tout se passe correctement, l'utilisateur est redirigé vers la page de résultats correspondant à sa session. En cas d'erreur durant l'envoi ou la sauvegarde, une notification d'erreur est affichée pour informer l'utilisateur sans bloquer l'interface. Toute la navigation, l'affichage et le suivi sont gérés en temps réel, offrant une expérience fluide et immersive pendant toute la durée du test.





KnowledgeQuest

Tableau de bord

Accueil

Créer un QCM

Charger un document

Passer un test

Statistiques

Paramètres

MB
Mouad Bentefrit
Médecine

Déconnexion

Question 2

X Incorrect

Quel virus cause la rougeole ?

Votre réponse : Herpèsvirus

Bonne réponse : Paramyxovirus

Question 3

X Incorrect

Quel parasite est à l'origine du paludisme ?

Votre réponse : Plasmodium falciparum

Bonne réponse : Toxoplasma gondii

Question 4

✓ Correct

Quelle levure est souvent responsable de candidose buccale ?

Votre réponse : Aspergillus fumigatus

Bonne réponse : Aspergillus fumigatus

KnowledgeQuest

Tableau de bord

Accueil

Créer un QCM

Charger un document

Passer un test

Statistiques

Paramètres

MB
Mouad Bentefrit
Médecine

Déconnexion

Résultats du test

Réalisé le 25/04/2025

20%
À AMÉLIORER

1 / 5 bonnes réponses

Durée : 0min 28s

Exporter PDF

Partager

Revoir erreurs

Nouveau test

Retour au tableau de bord

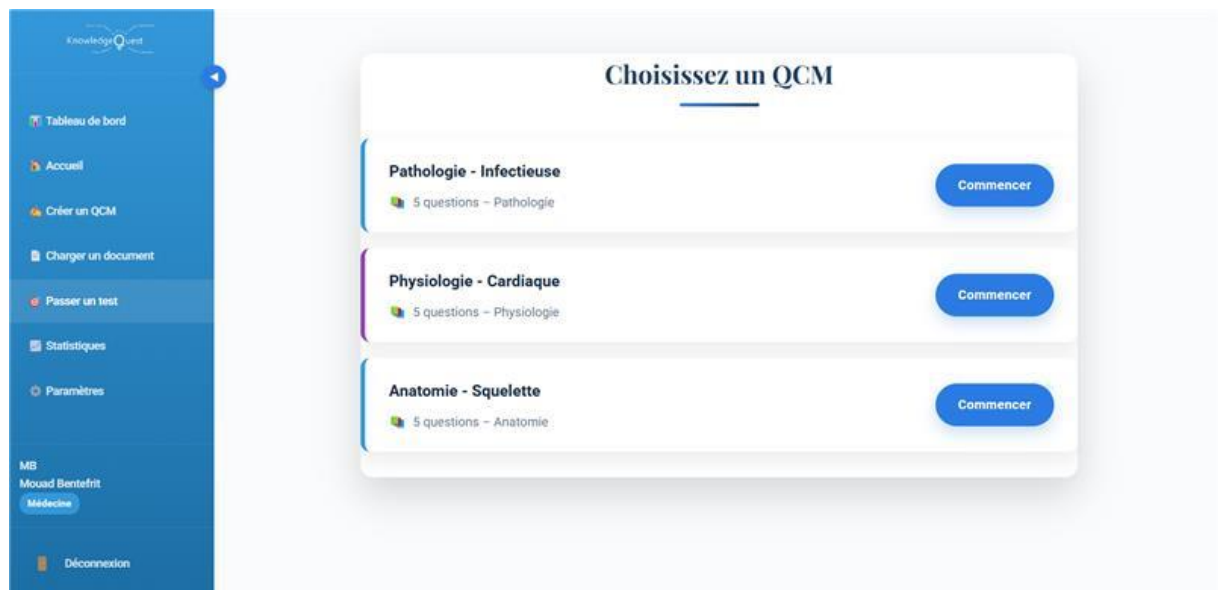
Revue des questions

Question 1

X Incorrect

Quel agent est responsable de la fièvre typhoïde ?

Votre réponse : Salmonella typhi



Page statistiques :

La page "Statistiques" de Knowledge Quest permet aux utilisateurs de suivre visuellement leur progression académique à travers plusieurs indicateurs détaillés. Lors de son ouverture, le script vérifie d'abord que l'utilisateur est bien connecté, garantissant la protection des données personnelles. Ensuite, l'application récupère les statistiques de l'utilisateur, notamment son historique de sessions de test, en interrogeant statsService. Si des données sont disponibles, la page affiche plusieurs résumés clés comme le score moyen, le nombre total de tests passés, ainsi que le meilleur et le pire score enregistré. Deux graphiques sont ensuite générés : l'évolution des scores au fil du temps et la répartition des matières étudiées. L'utilisateur peut également filtrer ces graphiques selon différentes périodes temporelles (toutes, dernier mois, dernière semaine), ce qui ajuste dynamiquement les données affichées sans recharger la page.

En dessous des statistiques globales, une table liste les dernières sessions réalisées, avec pour chaque session la date, le score obtenu, et un lien direct vers la page de résultats détaillés. Si aucun test n'a encore été effectué, la page propose une interface vide incitant l'utilisateur à commencer un test pour générer ses premières statistiques. Toute erreur lors du chargement des données est capturée et notifiée de façon non intrusive pour préserver une expérience utilisateur fluide et professionnelle.

Page paramètres :

La page "Paramètres" de Knowledge Quest permet aux utilisateurs de personnaliser leur expérience à travers une interface structurée en plusieurs onglets thématiques :



Interface, Notifications, Confidentialité, Compte, et Préférences d'étude. Chaque section regroupe des paramètres spécifiques sous forme de formulaires interactifs.

Dans l'onglet Interface, les utilisateurs peuvent activer le mode sombre, choisir la taille de police (petite, moyenne, grande) et sélectionner un thème de couleur en fonction de leur domaine d'étude (Médecine, Droit ou Neutre). L'onglet Notifications propose de gérer la réception d'emails pour les nouveautés, d'activer les rappels de révision, et d'ajuster leur fréquence (quotidienne, hebdomadaire, etc.).

Le volet Confidentialité donne la possibilité d'activer ou désactiver le partage de données anonymisées et de configurer la durée de conservation des documents téléchargés. L'onglet Compte offre des actions sensibles : l'exportation de toutes ses données utilisateur/QCMs et la suppression définitive du compte, avec les précautions visuelles nécessaires (zone rouge).

Enfin, l'onglet Préférences d'étude permet de définir le mode d'apprentissage favori (aléatoire, progressif, répétition espacée), la limite de temps par question, ainsi qu'un objectif quotidien en nombre de questions.

Toutes les modifications sont regroupées et sauvegardées en cliquant sur un unique bouton "Enregistrer les modifications", centralisant ainsi la gestion et la persistance des paramètres dans localStorage ou via appel API, pour garantir la continuité de l'expérience utilisateur sur toutes les sessions.

Page Profil :

La page Profil de l'application Knowledge Quest permet aux utilisateurs de consulter et modifier leurs informations personnelles. Lors de l'ouverture de la page, une vérification est effectuée pour s'assurer que l'utilisateur est connecté. Si ce n'est pas le cas, il est redirigé automatiquement vers la page de connexion pour des raisons de sécurité.

Une fois connecté, les informations de l'utilisateur sont extraites du stockage local et affichées sur l'interface : le nom complet, l'adresse email, le domaine d'étude (par exemple Médecine ou Droit), l'avatar choisi ainsi que la date d'inscription. Ces informations sont aussi préremplies dans le formulaire de modification de profil afin de faciliter l'édition.

La page est divisée en plusieurs onglets. Le premier onglet permet à l'utilisateur de modifier ses informations personnelles, telles que son prénom, son nom, son email, son domaine d'étude ainsi que son avatar. Lors de la soumission du formulaire, une requête est envoyée au serveur pour mettre à jour les données de l'utilisateur. Si la mise à jour est réussie, les nouvelles informations sont sauvegardées localement pour maintenir la session à jour.

Le deuxième onglet est consacré au changement du mot de passe. L'utilisateur doit saisir son mot de passe actuel ainsi qu'un nouveau mot de passe. Une validation est effectuée côté serveur afin de vérifier que l'ancien mot de passe est correct avant de procéder à la modification.

Le troisième onglet présente l'historique des tests passés. Pour cela, une requête est envoyée afin de récupérer les dernières sessions de test de l'utilisateur. Chaque session affiche le titre du QCM, le score obtenu et la date du test.



La gestion de l'avatar est également disponible directement depuis la section du profil. L'utilisateur peut choisir parmi une liste prédéfinie et voir son avatar mis à jour en temps réel dans l'interface.

Toutes ces fonctionnalités sont centralisées pour offrir à l'utilisateur un espace complet de gestion de ses informations et de suivi de ses performances sur Knowledge Quest.

Backend

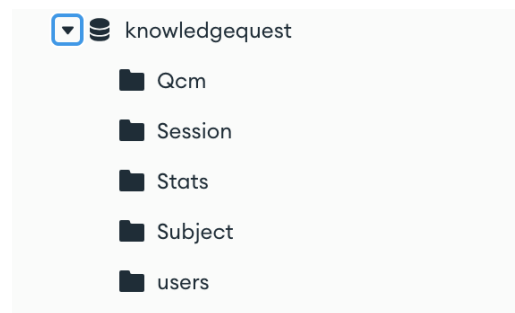
Le backend de mon application web assure plusieurs fonctionnalités essentielles telles que la gestion de l'authentification des utilisateurs, la création et la validation de sessions sécurisées, la communication avec la base de données pour l'enregistrement et la récupération des informations, ainsi que le traitement des différentes requêtes envoyées par le frontend. Il prend en charge la sécurisation des échanges, la vérification des données reçues, l'application des règles métier et la réponse aux différentes actions réalisées par les utilisateurs sur l'interface. Le backend est donc responsable de garantir la bonne circulation des informations entre les différentes couches de l'application tout en assurant la sécurité, la fiabilité et la performance du service.

Lancer le backend avec server.js :

Le fichier server.js est le point d'entrée principal du backend de l'application. Il commence par importer les principales dépendances nécessaires au fonctionnement du serveur, notamment express pour gérer les requêtes HTTP, mongoose pour la connexion à MongoDB, ainsi que d'autres modules comme dotenv pour la gestion des variables d'environnement, cors pour autoriser les requêtes cross-origin, helmet pour sécuriser les en-têtes HTTP, morgan pour le logging des requêtes, et compression pour améliorer les performances en compressant les réponses envoyées.

L'application Express est initialisée, puis configurée avec plusieurs middlewares destinés à renforcer la sécurité, autoriser les connexions de différentes origines, parser le corps des requêtes en JSON, compresser les réponses et afficher les requêtes HTTP dans la console pour faciliter le suivi des activités serveur. Un logger personnalisé est également ajouté pour afficher dans la console toutes les requêtes entrantes avec l'heure précise.

La connexion à MongoDB est ensuite établie à l'aide de l'URI stockée dans les variables d'environnement. Une confirmation de réussite ou un message d'erreur est affiché selon le résultat de la tentative de connexion. Le serveur est également configuré pour servir statiquement les fichiers présents dans le dossier uploads, notamment pour les avatars de profil utilisateur.



Le serveur importe ensuite toutes les routes de l'application, séparées par fonctionnalités : authentification, utilisateurs, QCMs, sessions, statistiques, matières, documents et paramètres. Chaque groupe de routes est monté sous un chemin d'API spécifique, garantissant une organisation claire et une accessibilité simple via l'URL de l'API.

Une route de test est définie sur `/api` pour vérifier rapidement si le serveur est opérationnel. Une gestion globale des erreurs est mise en place pour intercepter les erreurs non prises en charge ailleurs, renvoyer un message d'erreur structuré au client et afficher les détails en environnement de développement.

Enfin, le serveur est lancé en écoutant sur le port défini dans les variables d'environnement ou sur le port 5000 par défaut, avec un message de confirmation affiché dans la console. Le fichier exporte également l'application Express, permettant de la réutiliser facilement pour des tests ou d'autres opérations.

Les fichiers de routes définissent tous les points d'entrée de l'API qui permettent au frontend de communiquer avec le backend. Chaque fichier de route correspond à une fonctionnalité spécifique de l'application et regroupe les différentes URL accessibles pour manipuler les données associées. Le fichier `authRoutes` gère l'inscription, la connexion et la déconnexion des utilisateurs. `userRoutes` permet de récupérer, mettre à jour ou supprimer les informations personnelles des utilisateurs. `qcmRoutes` offre des routes pour créer, récupérer, modifier ou supprimer des QCMs. `sessionRoutes` permet d'enregistrer les sessions de réponse des utilisateurs et de consulter leur historique. `statsRoutes` expose les statistiques utilisateurs comme les scores ou les moyennes de performance. `subjectRoutes` est utilisé pour la gestion des matières disponibles dans l'application. `documentRoutes` gère l'upload, la consultation et la suppression des documents déposés par les utilisateurs. Enfin, `settingsRoutes` permet aux utilisateurs d'ajuster leurs paramètres personnels. Toutes ces routes sont protégées ou enrichies par des middlewares lorsque nécessaire, et elles centralisent la logique de réception des requêtes, déléguant ensuite leur traitement aux contrôleurs correspondants.

Session des utilisateurs :

Le modèle Session permet d'enregistrer et de suivre les sessions de réponse des utilisateurs lorsqu'ils complètent un QCM dans l'application. Chaque session est liée à un utilisateur spécifique via le champ `userId`, et à un questionnaire via le champ `qcmId`, tous



deux étant des références à leurs modèles respectifs. Cela permet de retracer précisément qui a répondu à quel QCM.

Pour chaque session, plusieurs informations essentielles sont stockées : le score obtenu par l'utilisateur à la fin du QCM, ainsi que la durée, qui correspond au temps total mis pour terminer le questionnaire, exprimé en secondes.

Le modèle contient aussi une liste détaillée appelée `questionsAnswered`, qui enregistre pour chaque question rencontrée : l'énoncé de la question, la réponse donnée par l'utilisateur (`userAnswer`), la bonne réponse (`correctAnswer`) et un booléen `isCorrect` indiquant si la réponse était correcte ou non. Cette structure permet de conserver l'historique précis de toutes les réponses et d'analyser en détail la performance de l'utilisateur question par question.

Grâce à l'option `timestamps`, la date et l'heure de création ainsi que de dernière mise à jour de chaque session sont automatiquement enregistrées, ce qui permet un suivi temporel des activités des utilisateurs.

Ce modèle joue un rôle central pour évaluer les progrès, générer des statistiques détaillées et proposer un retour personnalisé aux utilisateurs en fonction de leurs performances.

Le fichier `sessionRoutes.js` organise toutes les routes nécessaires à la gestion des sessions utilisateurs dans l'application. Toutes les routes sont protégées par le middleware `protect`, ce qui signifie qu'elles ne sont accessibles qu'aux utilisateurs connectés. La route `POST /` permet de créer une nouvelle session, généralement après qu'un utilisateur ait terminé un QCM, en enregistrant son score, son temps et ses réponses. La route `GET /:id` permet de récupérer les détails d'une session spécifique en fournissant son identifiant. La route `GET /user/:userId?` permet de récupérer toutes les sessions d'un utilisateur donné ; si aucun identifiant n'est fourni, elle retourne les sessions de l'utilisateur actuellement connecté, tandis qu'un administrateur peut accéder aux sessions de n'importe quel utilisateur. Ce fichier structure ainsi toutes les interactions nécessaires pour suivre, consulter et exploiter l'historique des activités des utilisateurs sur l'application.

Gestion des utilisateurs :

Le modèle `User` définit la structure des utilisateurs dans l'application. Chaque utilisateur possède un `name` et un `email` obligatoire. L'email est unique et doit correspondre à un format standard pour garantir sa validité. Le mot de passe n'est pas stocké directement ; c'est un `passwordHash` qui est enregistré après avoir été encrypté pour assurer la sécurité des informations sensibles. Ce hachage est géré automatiquement avant l'enregistrement grâce à un hook `pre-save` qui utilise la bibliothèque `bcryptjs`.

Chaque utilisateur appartient obligatoirement à un domaine défini parmi deux choix possibles : « Médecine » ou « Droit », ce qui permet d'adapter l'expérience utilisateur selon sa filière d'étude. La date de création du compte est enregistrée automatiquement à la création avec le champ `createdAt`.

Le modèle prévoit aussi une section `settings` pour permettre aux utilisateurs de personnaliser leur expérience. Ils peuvent choisir leur thème, la taille de la police, activer ou



désactiver le mode sombre, gérer les notifications par email, configurer la fréquence des rappels, ajuster leurs préférences de confidentialité concernant le partage de données et la durée de conservation des documents. Des options spécifiques aux habitudes d'étude sont également prévues, comme le mode d'étude (« aléatoire » par défaut), le temps accordé par question et l'objectif quotidien de questions à traiter.

Enfin, une méthode `matchPassword` est disponible pour comparer un mot de passe fourni par l'utilisateur au mot de passe haché stocké dans la base de données, ce qui permet de vérifier l'identité de manière sécurisée au moment de la connexion. Ce modèle est donc central pour l'authentification, la gestion des profils et la personnalisation de l'expérience utilisateur dans l'application.

Le fichier `userRoutes.js` définit toutes les routes liées à la gestion des utilisateurs dans l'application. Toutes ces routes sont protégées par le middleware `protect`, ce qui signifie qu'elles sont accessibles uniquement aux utilisateurs authentifiés. La route `GET /me` permet de récupérer les informations personnelles de l'utilisateur connecté. La route `PUT /profile` permet de mettre à jour les données du profil, comme le nom ou l'email. La route `PUT /domain` permet à l'utilisateur de changer son domaine d'étude, par exemple passer de Droit à Médecine. La route `PUT /password` permet de modifier son mot de passe en toute sécurité. La route `GET /test-history` permet de consulter l'historique des tests et sessions réalisées. Une route spécifique `GET /all` est également présente pour les administrateurs, permettant de récupérer la liste complète de tous les utilisateurs enregistrés dans l'application. Ces routes centralisent ainsi toutes les interactions possibles avec les données utilisateur côté serveur.

Choix des matières :

Le modèle `Subject` définit les matières disponibles dans l'application. Chaque matière possède un `name` obligatoire qui représente son intitulé, par exemple « Anatomie » ou « Droit civil ». Chaque matière est aussi associée à un `domain`, qui précise si elle appartient au cursus de « Médecine » ou de « Droit », avec une contrainte pour n'accepter que ces deux valeurs possibles, garantissant ainsi la cohérence des données.

En plus de son nom et de son domaine, chaque matière contient une liste de `topics`, c'est-à-dire des sous-thèmes ou des chapitres abordés dans la matière. Cette liste est obligatoire et permet de structurer plus finement le contenu pédagogique proposé aux utilisateurs.

Grâce à cette organisation, le modèle `Subject` permet de classer les QCMs, les documents ou les parcours d'étude en fonction du domaine de l'utilisateur et des matières qu'il choisit de travailler, offrant une navigation claire et adaptée dans l'application.

Le fichier `subjectRoutes.js` gère toutes les routes liées aux matières et aux sujets disponibles dans l'application. Certaines routes sont publiques et accessibles sans authentification, permettant notamment de récupérer toutes les matières existantes (`GET /all`), les `topics` associés à une matière spécifique (`GET /topics/:name`), les matières filtrées par domaine (`GET /domain/:domain`), ou encore de récupérer une matière précise par son nom (`GET /:name`). Après l'application du middleware `protect`, qui impose l'authentification, d'autres



routes deviennent accessibles uniquement aux utilisateurs connectés. La route GET / permet aux utilisateurs de récupérer tous les topics disponibles en fonction de leur propre domaine d'étude. Deux routes sont réservées aux administrateurs ou aux utilisateurs autorisés : POST / pour créer une nouvelle matière et PUT /:name pour modifier une matière existante. Ce fichier organise donc l'accès, la création et la mise à jour des matières en fonction du niveau d'accès de l'utilisateur.

Gestions des statistiques :

Le modèle Stats permet de suivre l'évolution des performances des utilisateurs dans l'application. Chaque document de statistiques est lié à un utilisateur spécifique via le champ `userId`, qui est une référence au modèle User. Cela permet d'associer facilement les résultats de performance à l'identité de chaque utilisateur.

Le champ `scoresHistory` est un tableau qui enregistre l'historique des scores obtenus par l'utilisateur. Chaque entrée de cet historique contient une date marquant le moment où le score a été enregistré et le score correspondant. Ce mécanisme permet de suivre l'évolution des résultats dans le temps et d'observer la progression ou les baisses de performance.

Le modèle contient aussi un champ `averageScore`, qui représente la moyenne des scores de l'utilisateur sur l'ensemble de ses sessions ou évaluations. Ce champ est initialisé à zéro et peut être mis à jour dynamiquement à mesure que l'utilisateur effectue de nouveaux QCMs.

Grâce à l'option `timestamps`, chaque enregistrement conserve automatiquement les dates de création et de dernière mise à jour, ce qui facilite l'analyse des données à long terme. Le modèle Stats est donc essentiel pour proposer aux utilisateurs des bilans personnalisés, des graphiques d'évolution ou des conseils basés sur leur progression.

Le fichier `statsRoutes.js` gère les routes permettant d'accéder aux statistiques des utilisateurs. Toutes les routes sont protégées par le middleware `protect`, ce qui signifie qu'elles sont réservées aux utilisateurs connectés. La route GET /me permet à un utilisateur de récupérer ses propres statistiques, notamment son historique de scores et sa moyenne générale. La route GET /:userId permet à un administrateur ou à un utilisateur autorisé de consulter les statistiques d'un autre utilisateur en fournissant son identifiant. Ce fichier organise donc l'accès aux données de performance et d'évolution des utilisateurs tout en garantissant la sécurité et la confidentialité de ces informations.

Gestion des QCM :

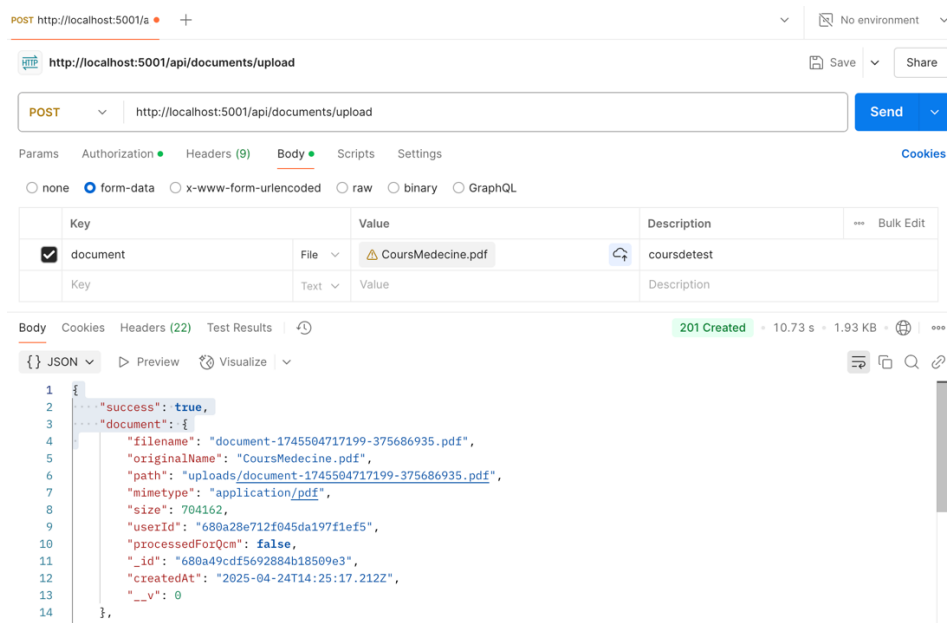
Le modèle Qcm définit la structure des questionnaires à choix multiples disponibles dans l'application. Chaque QCM possède un titre obligatoire qui correspond à son nom ou son thème général, ainsi qu'un `subject` qui indique la matière associée, permettant ainsi de classer les questionnaires selon les matières étudiées.



Le champ questions contient un tableau de questions. Chaque question est structurée avec son énoncé (question), une liste de choix de réponses possibles (choices), et la réponse correcte (correctAnswer). Tous ces champs sont obligatoires pour garantir que chaque question soit complète et utilisable lors des sessions de test.

Chaque QCM est aussi relié à un utilisateur via le champ createdBy, qui fait référence au modèle User. Cela permet d'identifier l'auteur du questionnaire, qu'il s'agisse d'un administrateur, d'un professeur ou d'un créateur de contenu autorisé.

Le champ createdAt est automatiquement renseigné avec la date et l'heure de création du QCM grâce à une valeur par défaut. Ce modèle est essentiel pour alimenter les évaluations proposées aux utilisateurs, pour suivre la provenance des contenus créés, et pour gérer l'organisation des QCMs dans l'application par matière et par auteur.



Le fichier qcmRoutes.js définit toutes les routes nécessaires à la gestion des QCMs dans l'application. Toutes les routes sont protégées par le middleware protect, ce qui signifie que seuls les utilisateurs authentifiés peuvent y accéder. La route POST / permet de créer un nouveau QCM, tandis que GET / permet de récupérer tous les QCMs existants. Pour un QCM spécifique, la route GET /:id permet d'obtenir ses détails, la route PUT /:id permet de le modifier, et la route DELETE /:id permet de le supprimer. Une route pour générer un QCM automatiquement à partir d'un document était également prévue (POST /generate), mais elle est actuellement commentée. Ce fichier structure donc toutes les opérations de création, lecture, modification et suppression des QCMs, en assurant un accès sécurisé et organisé aux questionnaires de l'application.



Gestions des documents :

Le modèle Document permet de gérer les fichiers que les utilisateurs téléchargent dans l'application. Chaque document enregistré possède plusieurs informations obligatoires : le filename qui correspond au nom du fichier sur le serveur, le originalName qui est le nom d'origine du fichier uploadé, le path qui indique son emplacement de stockage, le mimetype qui précise le type de fichier (par exemple PDF, Word, etc.), ainsi que la size qui enregistre la taille du fichier en octets.

Chaque document est associé à un utilisateur via le champ userId, une référence au modèle User, ce qui permet de retrouver facilement tous les fichiers envoyés par un utilisateur donné.

Le modèle prévoit également un champ processedForQcm, qui indique si le document a été analysé et transformé en QCM. Si c'est le cas, le champ generatedQcmId contient une référence vers le QCM qui a été généré à partir de ce document, permettant de lier automatiquement les ressources éducatives et les évaluations.

La date d'enregistrement du fichier est conservée grâce au champ createdAt avec une valeur par défaut correspondant à la date de création. Ce modèle est donc essentiel pour organiser les documents utilisateurs, permettre leur traitement automatique, et enrichir la base de données de contenus pédagogiques directement exploitables dans l'application.

Le fichier documentRoutes.js organise toutes les routes nécessaires à la gestion des documents utilisateurs dans l'application. Toutes les routes sont protégées par le middleware protect, ce qui garantit que seul un utilisateur connecté peut uploader ou télécharger des documents. La route POST /upload permet aux utilisateurs d'envoyer un document vers le serveur ; cette route utilise d'abord un middleware pour gérer l'upload du fichier, puis déclenche un traitement du document après son envoi. La route GET /download/:filename permet de télécharger un fichier en fournissant son nom. Ce fichier assure ainsi la gestion sécurisée des fichiers utilisateurs, depuis leur envoi jusqu'à leur récupération.



API

L'API joue un rôle central dans le projet en assurant la communication entre le frontend et le backend de l'application. Elle permet aux différentes fonctionnalités de l'interface utilisateur d'interagir avec la base de données de manière sécurisée et structurée. À travers des points d'accès dédiés, l'API gère l'authentification des utilisateurs, la récupération et la mise à jour des informations personnelles, la création et la consultation des QCMs, l'upload de documents, l'enregistrement des sessions de test ainsi que le suivi statistique de la progression des utilisateurs. L'API garantit ainsi une séparation claire entre la logique de présentation et la logique de traitement des données, tout en offrant une expérience utilisateur fluide et dynamique.

Le module Utilisateur gère l'ensemble des opérations liées au profil des utilisateurs de l'application. Un utilisateur connecté peut consulter ses propres informations personnelles via l'API `/users/me`, mettre à jour son profil (nom, email) avec `/users/profile`, ou modifier son mot de passe grâce à la route `/users/password`. L'utilisateur peut également récupérer son historique de tests passés avec `/users/test-history`, lui permettant de suivre ses activités dans l'application. Enfin, à travers `/settings`, il peut consulter et mettre à jour ses paramètres personnels tels que le thème de l'interface, la fréquence des notifications ou ses objectifs d'étude quotidiens.

Le module QCMs est dédié à la gestion des questionnaires à choix multiples proposés aux utilisateurs. Il permet de récupérer l'ensemble des QCMs disponibles grâce à `/qcms`, d'accéder aux détails d'un QCM spécifique via `/qcms/:id`, de créer un nouveau QCM avec `/qcms`, de mettre à jour un QCM existant en utilisant `/qcms/:id`, ou encore de supprimer un QCM avec la même route en méthode `DELETE`. Ce système assure la flexibilité nécessaire pour enrichir et maintenir la base de questionnaires proposés aux étudiants, tout en donnant aux administrateurs la possibilité de gérer dynamiquement le contenu pédagogique.

Le module Documents permet aux utilisateurs d'uploader leurs propres fichiers pédagogiques, notamment sous les formats PDF, Word ou TXT. L'upload d'un document est réalisé via la route `/documents/upload`, où le fichier est ensuite traité et enregistré sur le serveur. Les documents peuvent ensuite être récupérés par leur nom en utilisant la route `/documents/download/:filename`. Ce module facilite ainsi l'ajout et la gestion de ressources personnalisées pour les utilisateurs, en complément du contenu de l'application.

Le module Sessions est responsable de l'enregistrement et du suivi des tentatives de QCMs des utilisateurs. Lorsqu'un test est terminé, une nouvelle session est créée via `/sessions`, enregistrant des informations comme le score obtenu, la durée et le détail des réponses fournies. Il est possible de consulter les informations d'une session spécifique avec `/sessions/:id` ou de récupérer l'ensemble de ses sessions passées en utilisant `/sessions/user`. Ce suivi détaillé permet à chaque utilisateur de visualiser son historique de tests et d'analyser sa progression dans le temps.

Le module Statistiques permet aux utilisateurs de suivre leur évolution de manière synthétique. En utilisant `/stats`, un utilisateur connecté peut consulter ses propres statistiques personnelles, comprenant notamment sa moyenne de scores et



son historique de performance. De plus, l'API propose une route `/stats/aggregated/data` pour récupérer des données agrégées, utiles notamment pour l'affichage de tableaux de bord globaux ou pour une analyse à plus grande échelle des résultats au sein de l'application.

Enfin, le module Matières et sujets assure la gestion de l'organisation thématique des contenus de l'application. Il est possible de récupérer toutes les matières associées au domaine de l'utilisateur avec `/subjects`, ou de consulter directement une matière précise par son nom via `/subjects/:name`. La route `/subjects/domain/:domain` permet de filtrer les matières selon un domaine spécifique, comme Médecine ou Droit. Les administrateurs peuvent également enrichir l'application en créant de nouvelles matières via `/subjects`. Ce module garantit ainsi une classification logique et adaptée du contenu, facilitant la navigation et la recherche pour les utilisateurs.

Les contrôleurs jouent un rôle central dans l'architecture du backend en faisant le lien entre les routes et la logique métier de l'application. Chaque contrôleur contient les fonctions qui définissent précisément ce que doit faire le serveur lorsqu'une route est appelée. Par exemple, le `authController` gère toutes les opérations liées à l'authentification comme l'inscription, la connexion ou la déconnexion des utilisateurs. Le `userController` s'occupe de la récupération, de la mise à jour des profils et du traitement des paramètres personnels. Le `qcmController` prend en charge la création, la récupération, la modification et la suppression des QCMs. Le `sessionController` gère l'enregistrement des résultats de tests et l'historique des sessions utilisateurs. Le `statsController` est responsable du calcul et de la mise à jour des statistiques de performance. Le `subjectController` contrôle l'accès aux matières et aux topics disponibles dans l'application. Enfin, le `documentController` gère l'upload, le traitement et le téléchargement des documents utilisateurs. Chaque contrôleur isole ainsi la logique propre à une fonctionnalité donnée, ce qui permet d'avoir un code organisé, clair, et facile à maintenir.

L'API d'IA intégrée dans le projet joue un rôle clé en permettant la génération automatique de QCMs à partir de documents envoyés par les utilisateurs. Lorsqu'un utilisateur télécharge un fichier au format PDF, Word ou TXT, le serveur lit le contenu du document et en extrait le texte brut. Ce texte est ensuite utilisé pour formuler une requête destinée à l'API d'OpenAI, plus précisément au modèle GPT-4o-mini. À travers un prompt précis, l'IA est chargée de générer un QCM composé de cinq questions, chacune avec quatre choix de réponse, et de fournir ce contenu sous forme de JSON structuré. Le backend récupère alors cette réponse, la nettoie si nécessaire, la convertit en objet JSON exploitable, et la renvoie au frontend. Cette automatisation permet de transformer des documents pédagogiques classiques en questionnaires interactifs, enrichissant ainsi l'application de manière rapide et intelligente sans nécessiter de saisie manuelle de la part des utilisateurs. L'intégration de l'IA améliore considérablement l'expérience utilisateur en rendant la création de QCMs plus simple, plus rapide et plus accessible à tous.



```
Params  Authorization  Headers (9)  Body  Scripts  Settings
Body  Cookies  Headers (22)  Test Results  201 Created
{} JSON  Preview  Visualize
36      "correctAnswerIndex": 3
37      },
38      {
39          "question": "Quel type d'articulation permet le plus de mouvement ?",
40          "choices": [
41              "Articulation synoviale",
42              "Articulation fibreuse",
43              "Articulation cartilagineuse",
44              "Articulation plane"
45          ],
46          "correctAnswerIndex": 0
47      },
48      {
49          "question": "Quelle est la principale fonction des nerfs spinaux ?",
50          "choices": [
51              "Contrôler l'activité musculaire volontaire",
52              "Réguler la pression artérielle",
53              "Fournir des sensations cutanées",
54              "Relayer les signaux auditifs"
55          ],
56          "correctAnswerIndex": 2
57      },
58      {
59          "question": "Quel mécanisme est responsable de la régulation du pH sanguin ?",
60          "choices": [
61              "Système nerveux autonome",
62              "Système tampon de bicarbonate",
63              "Hématopoïèse",
64              "Muscle cardiaque"
65          ],
66          "correctAnswerIndex": 1
67      }
}
```

L'application propose une authentification étendue permettant aux utilisateurs de se connecter via leur compte Google ou Microsoft, en plus de l'authentification classique par email et mot de passe. Pour l'authentification Google, l'application utilise le service Google Identity Services, qui initialise un bouton de connexion directement sur l'interface utilisateur. Lorsqu'un utilisateur clique sur ce bouton, une authentification OAuth s'ouvre et, après validation, Google renvoie un token d'identité sécurisé. Ce token est ensuite envoyé au backend, où il est vérifié par le serveur à l'aide de Firebase Admin pour confirmer l'authenticité du compte. Si l'email associé existe déjà dans la base de données, l'utilisateur est connecté automatiquement ; sinon, un nouveau compte est créé avec les informations du profil Google comme le nom, l'adresse email et une image de profil si disponible. Le backend génère ensuite un token JWT pour sécuriser la session.

Concernant l'authentification Microsoft, l'application utilise le protocole OAuth 2.0 de Microsoft. Lorsqu'un utilisateur choisit l'option Microsoft, il est redirigé vers la page d'authentification officielle de Microsoft où il doit valider son identité. Après la connexion, Microsoft renvoie un code d'autorisation que le backend échange contre un token d'accès. Grâce à ce token, le serveur interroge l'API Microsoft Graph pour obtenir les informations de l'utilisateur, telles que son nom complet et son adresse email. Si l'utilisateur est déjà présent dans la base de données, il est connecté ; sinon, un nouveau compte est automatiquement créé avec les informations fournies par Microsoft. Comme pour Google, un token JWT est généré pour maintenir la session utilisateur.

Ces deux méthodes d'authentification offrent une alternative rapide et sécurisée aux utilisateurs, évitant la saisie manuelle de mots de passe et s'appuyant sur des infrastructures de sécurité éprouvées. Elles permettent également de simplifier l'inscription en réduisant le



nombre d'étapes nécessaires à la création d'un compte, tout en assurant une intégration fluide et fiable avec le backend de l'application.

4. Tests et Validation

Stratégie de Test

La phase de test a été essentielle pour garantir la stabilité et la performance de l'application. Nous avons défini une stratégie de test structurée en trois niveaux : tests unitaires, tests d'intégration, et tests système.

Les tests unitaires ont été réalisés sur les fonctions critiques du backend, notamment celles en charge de la communication avec l'API d'intelligence artificielle, du traitement des documents et de la création des objets QCM. Ces tests ont permis de s'assurer que chaque composant fonctionnait de manière isolée et conforme aux attentes.

Les tests d'intégration ont porté sur la vérification de la communication entre les différentes couches de l'application. Nous avons vérifié que les données envoyées depuis le frontend étaient correctement transmises au backend, traitées, puis retournées et affichées de manière cohérente à l'utilisateur. Postman a été utilisé pour simuler les requêtes API et observer les réponses du serveur.

Enfin, les tests système ont consisté à valider l'ensemble du processus du point de vue de l'utilisateur final. Nous avons testé différents parcours utilisateur : envoi de document, génération automatique de QCM, participation à un quiz, et consultation des résultats. Ces tests ont été effectués manuellement pour détecter d'éventuels dysfonctionnements ou incohérences dans l'interface utilisateur.

Résultats Obtenus

Les tests ont permis de valider la majorité des fonctionnalités attendues. Le système de génération de QCM a prouvé sa robustesse même en présence de documents longs ou complexes. Les QCM produits étaient cohérents et variés, et les réponses bien structurées.

L'interface de quiz a réagi correctement aux interactions de l'utilisateur, avec un bon retour visuel et une fluidité d'affichage. Les statistiques de performance ont été affichées sans erreur et correctement mises à jour après chaque session.

Quelques bugs mineurs ont été identifiés lors des tests système, notamment des cas particuliers de fichiers mal formatés ou des problèmes d'alignement d'éléments sur mobile. Ces points ont été corrigés dans les dernières phases de développement.



Identification et Correction des Erreurs

Parmi les erreurs rencontrées, on peut citer :

- Des conflits dans le parsing des fichiers DOCX contenant des tableaux complexes.
- Des erreurs HTTP 500 causées par des fichiers corrompus non pris en charge.
- Des réponses non formatées correctement en JSON, nécessitant une validation plus stricte côté backend.

Pour chaque bug identifié, nous avons mis en place une procédure de diagnostic avec journalisation des erreurs et messages explicites côté client. L'ajout de try/catch généralisés et la mise à jour des dépendances Node.js ont permis de renforcer la résilience du backend.

Améliorations Apportées suite aux Tests

Les tests nous ont permis d'apporter plusieurs améliorations clés :

- Optimisation de l'algorithme de tri des questions générées.
- Refonte partielle de l'affichage des résultats avec une meilleure hiérarchisation visuelle.
- Meilleure prise en charge des erreurs grâce à des messages clairs et contextuels.
- Ajout d'un indicateur de chargement pendant les appels à l'IA pour améliorer l'expérience utilisateur.

5. Bilan et Perspectives

Bilan du Travail

Le projet Knowledge Quest a été mené à bien dans le respect des objectifs fixés initialement. Nous avons conçu et développé une application web fonctionnelle, dotée d'une interface intuitive et d'un backend robuste, capable de transformer automatiquement des documents pédagogiques en QCM grâce à l'intelligence artificielle.

L'ensemble des fonctionnalités principales est opérationnel : téléversement de fichiers, génération de questionnaires, participation à des quiz, et suivi des performances. L'architecture logicielle s'est révélée adaptée et scalable, et le projet a été conduit selon une méthodologie agile, avec un bon rythme de progression et une collaboration efficace au sein de l'équipe.



Défis Relevés

L'un des principaux défis techniques rencontrés concernait l'intégration de l'API d'intelligence artificielle, notamment la gestion des réponses incohérentes ou incomplètes. Cela nous a amenés à développer des mécanismes de filtrage et de validation du contenu généré, afin de garantir une qualité pédagogique minimale.

Le traitement des fichiers DOCX et PDF a également présenté des complexités, liées à la diversité des formats et structures possibles. Nous avons dû ajuster notre pipeline de parsing pour assurer une extraction fiable du texte, quel que soit le style du document.

Sur le plan organisationnel, il a fallu gérer la répartition des tâches et les disponibilités de chacun, notamment dans un contexte d'alternance. Grâce à une communication fluide et à l'usage d'outils comme Trello et Discord, nous avons su maintenir une bonne coordination.

Intérêt du Projet pour l'Apprentissage

Ce projet nous a permis de consolider de nombreuses compétences techniques essentielles au développement web moderne. Nous avons appris à concevoir une application full-stack, à manipuler des API externes, à structurer une base de données NoSQL, et à gérer la communication entre les différentes couches d'une application.

Au-delà de l'aspect technique, nous avons aussi acquis de l'expérience en gestion de projet, en travail collaboratif, et en résolution de problèmes concrets. Ce projet a été l'occasion d'appliquer des connaissances vues en cours dans un contexte réaliste et stimulant.

Difficultés Rencontrées

Nous avons rencontré plusieurs difficultés tout au long du développement. Certaines étaient techniques, comme les erreurs de parsing ou les contraintes liées aux appels API. D'autres relevaient de la conception UX/UI, où il a fallu faire des choix simples mais efficaces pour garantir une bonne expérience utilisateur.

Ces obstacles ont été surmontés grâce à une répartition claire des responsabilités, à une forte capacité d'adaptation, et à l'utilisation d'outils de test et de validation pour fiabiliser chaque brique du projet.

Leçons Apprises

Parmi les leçons tirées, l'importance d'une planification réaliste et de cycles de test réguliers ressort nettement. Nous avons compris que les tests ne doivent pas être une étape finale,



mais un processus continu et intégré dès le début du développement. Nous avons également pris conscience de l'impact de l'expérience utilisateur sur l'adoption d'une application, ce qui nous a incités à soigner l'interface et les retours visuels.

Perspectives d'Amélioration

Pour l'avenir, plusieurs axes d'amélioration sont envisagés. Nous pourrions par exemple :

- Intégrer un système d'authentification plus complet (OAuth, JWT).
- Ajouter des fonctionnalités de partage de QCM entre utilisateurs.
- Proposer une API publique pour permettre à d'autres applications d'utiliser notre moteur de génération de QCM.
- Mettre en place des tests automatisés pour faciliter les mises à jour.
- Améliorer le design responsif pour une expérience optimale sur mobile.

Évolutions Possibles du Projet

Knowledge Quest pourrait évoluer en une véritable plateforme d'apprentissage collaboratif. Il serait envisageable d'ajouter une dimension communautaire, avec la possibilité de noter les QCM, de commenter, ou même de créer des groupes de travail. L'intégration de modules adaptatifs basés sur les performances passées des utilisateurs pourrait également rendre la révision plus personnalisée et efficace.

En conclusion, ce projet a été une expérience riche à la fois sur le plan technique, humain et pédagogique. Il a renforcé notre motivation à développer des solutions utiles, innovantes et ancrées dans des besoins réels.

6. Conclusion

Conclusion

Le projet Knowledge Quest s'est inscrit dans une dynamique d'apprentissage intensif, conjuguant les compétences techniques, la gestion de projet, la collaboration et l'innovation. Grâce à ce travail, nous avons été amenés à concevoir une solution concrète à une problématique réelle rencontrée par les étudiants : l'optimisation du temps de révision à travers la génération automatique de QCM.

L'ensemble du processus, depuis la définition des besoins jusqu'à la mise en production d'un prototype fonctionnel, nous a permis de mettre en application nos acquis en JavaScript, en



architecture web et en intégration d'API. Nous avons su développer une application complète, full-stack, avec une interface fluide et une logique serveur performante.

Ce projet a été une opportunité précieuse pour affiner notre rigueur, notre organisation et notre capacité à travailler efficacement en équipe. Il a également renforcé notre motivation à contribuer à des projets technologiques utiles, porteurs de sens, et tournés vers l'amélioration de l'expérience utilisateur.

En somme, Knowledge Quest représente une réussite sur les plans technique, pédagogique et humain. Il constitue une base solide pour de futures évolutions et une preuve concrète de notre capacité à mener à bien un projet ambitieux de bout en bout.